影像處理 Image Processing

大學智慧機器人學程 | 高中 AI 機器人課程 | 台灣自編大學教科書

編者：李世淵(機器人叫獸)|助教：李天宇、李宇晴

第 9 章 影像分割

本章學習目標

- 說明影像分割在視覺流程中的定位，區分基於不連續性與基於相似性的兩大分割策略。
- 運用全域、Otsu 與自適應門檻法完成灰階影像的二值化，並判斷各自的適用時機。
- 推導並實作 Sobel、Laplacian of Gaussian 與 Canny 邊緣偵測，理解其中的低通與高通運算。
- 以霍夫轉換偵測影像中的直線與圓，說明「參數空間投票」的核心思想。
- 運用區域成長與分水嶺法完成基於相似性的分割，並以標記控制避免過度分割。
- 比較傳統分割與深度學習語意分割的差異，並說明分割成效對後續量測與辨識的影響。

叫獸開講|霍夫轉換：從次原子粒子軌跡到你家的車道偏離警示

1950 年代，粒子物理學面臨一個「幸福的煩惱」。1952 年，Donald Glaser 發明了氣泡室(Bubble Chamber)——讓帶電粒子通過過熱液體時，沿途激發出一串微小氣泡，形成可拍攝的軌跡，這項發明讓他拿下 1960 年諾貝爾物理獎。問題是：氣泡室每天產生成千上萬張底片，每一張都佈滿縱橫交錯的粒子軌跡，而當年只能靠人工用眼睛一條一條描出這些軌跡、量測曲率以推算粒子動量。資料產出的速度，遠遠超過人力分析的速度。

在密西根大學與 Glaser 共事的物理學家 Paul Hough，決定讓機器來做這件

苦差事。他要解決的核心難題是：如何讓電腦自動從一堆散亂的亮點中，找出「這些點排成了一條直線」?他的靈感堪稱神來一筆：與其在影像空間裡辛苦地嘗試各種直線去湊點，不如把問題翻轉到「參數空間」。影像中的每一個亮點，都對應到參數空間中的一條曲線(代表所有可能通過該點的直線);若許多亮點共線，它們對應的曲線就會在參數空間中交會於同一點——於是「找直線」這個困難的幾何問題，變成了「找參數空間中的交會熱點」這個單純的投票統計問題。1962 年，Hough 以「辨識複雜圖樣的方法與裝置」為題取得美國專利。

這個為高能物理而生的方法，經過 Rosenfeld 在 1969 年引入主流電腦視覺、以及 Duda 與 Hart 在 1972 年改用極座標表示法(解決了垂直線斜率無限大的問題)之後，成為影像處理最經典的工具之一，至今累積超過兩千五百篇相關論文。而它最貼近你我生活的化身，就在汽車裡：當代車輛的車道偏離警示系統，核心正是用霍夫轉換即時偵測路面上的車道線;當系統發現車輛正在無預警地偏離兩條偵測到的直線，便發出警報。一個誕生於粒子加速器的數學，六十年後守護著高速公路上每一位駕駛的安全。

這個故事的啟示，是本章的核心心法：分割的本質，常常是一次「觀點的翻轉」。把「找直線」翻轉成「投票統計」，把「切割區域」翻轉成「淹水分界」，把「哪裡是邊界」翻轉成「哪裡不連續」——換一個空間、換一種提問方式，難題就迎刃而解。這與第 6 章「換到頻率域」、第 7 章「換到參數空間解逆問題」的精神一脈相承。

思考題：霍夫轉換把「找直線」變成「在參數空間找熱點」。這個「投票」的思想非常強大——你能想到生活中還有哪些問題，也可以用「每個證據都投一票、票多者當選」的方式來解決?

9.1 什麼是影像分割

9.1.1 分割的定位：承上啟下的轉捩點

影像分割(Image Segmentation)的目標，是把影像劃分成若干個有意義的區域——把前景從背景中分離、把每個物件圈出來、把不同的組織或材質區分開。它是影像處理流程中一個關鍵的轉捩點：在它之前(第 1 到 8 章)，我們處理的是「像素」，做的是強化、還原、色彩調整這類「像素進、像素出」的低階運算;從分割開始，我們的關注點從「像素」升級到「區域」與「物件」，踏入了中階視覺的領域，並為第 13 章的特徵擷取、第 16 章的物件偵測鋪路。



圖 9-1　分割站在低階與中階視覺的交界上

這個轉變的重要性怎麼強調都不為過。回顧第 1 章提過的低、中、高階視覺三層架構：分割正是連接低階(像素)與高階(語意)的橋梁。一張影像若沒有經過分割，電腦看到的永遠只是一大片數字;唯有把「這一塊像素屬於同一個物件」界定出來，後續的「這個物件是什麼」才有意義。

9.1.2 兩大策略：不連續與相似

影像分割的方法雖然繁多，但都可以歸入兩大哲學：

1. 基於不連續性(Discontinuity)：著眼於「區域之間的界線」。它假設不同區域的交界處，像素值會發生劇烈變化，因此設法找出這些劇變的位置——這就是邊緣偵測(9.3 節)與霍夫轉換(9.4 節)的思路。用一句話說：找出「哪裡變了」，變的地方就是邊界。

2. 基於相似性(Similarity)：著眼於「區域內部的一致性」。它假設同一個區域內的像素，在某種性質(灰階、顏色、紋理)上是相似的，因此設法把相似的像素聚集成群——這就是門檻法(9.2 節)、區域成長與分水嶺法(9.5 節)的思路。用一句話說：找出「哪些像素是一夥的」，一夥的就是同一區域。

這兩種策略是一體兩面：區域的邊界，正是相似性中斷之處;而相似的內部，正是被邊界包圍的範圍。實務上，強健的分割系統常常兩者並用——先用邊緣界定大致輪廓，再用相似性填補區域，或反之。理解這個二分法，就掌握了本章的骨架。

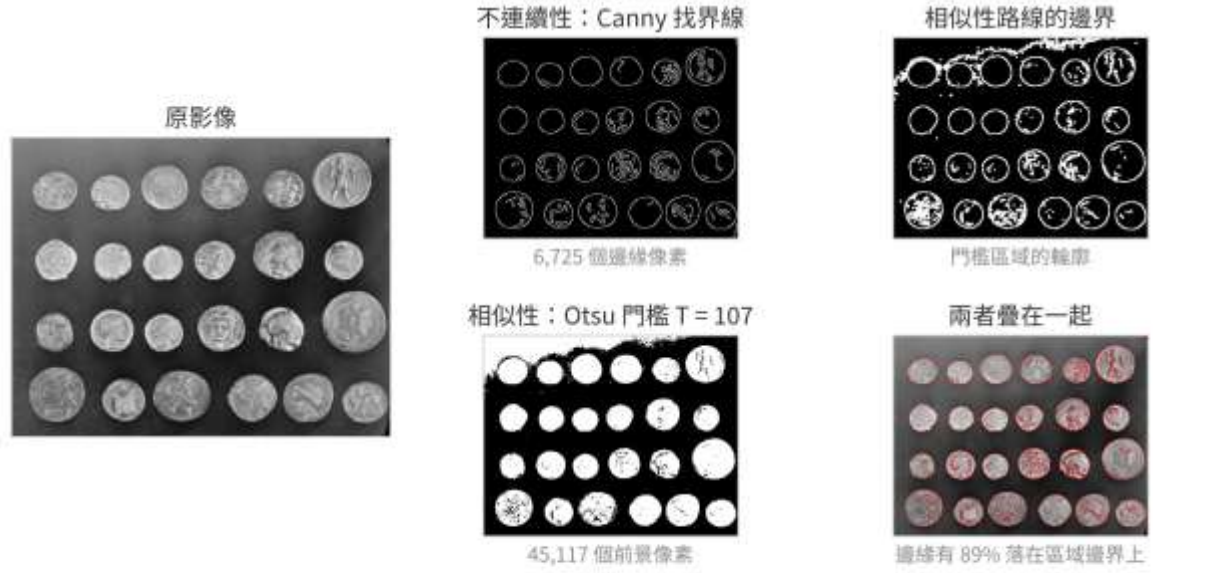


圖 9-2　同一張影像，兩條路線，指的是同一條界線

9.1.3 本章在全書中的位置

分割是承先啟後的樞紐，幾乎用到前八章的所有工具，也支撐著後續各章：

- 與第 5 章(空間域強化)：分割前的品質決定分割的成敗。對比不足的影像要先做直方圖等化或 CLAHE，否則門檻分不開前景背景;雜訊會讓邊緣偵測產生大量假邊，因此 Canny 的第一步就是第 5 章的高斯平滑。

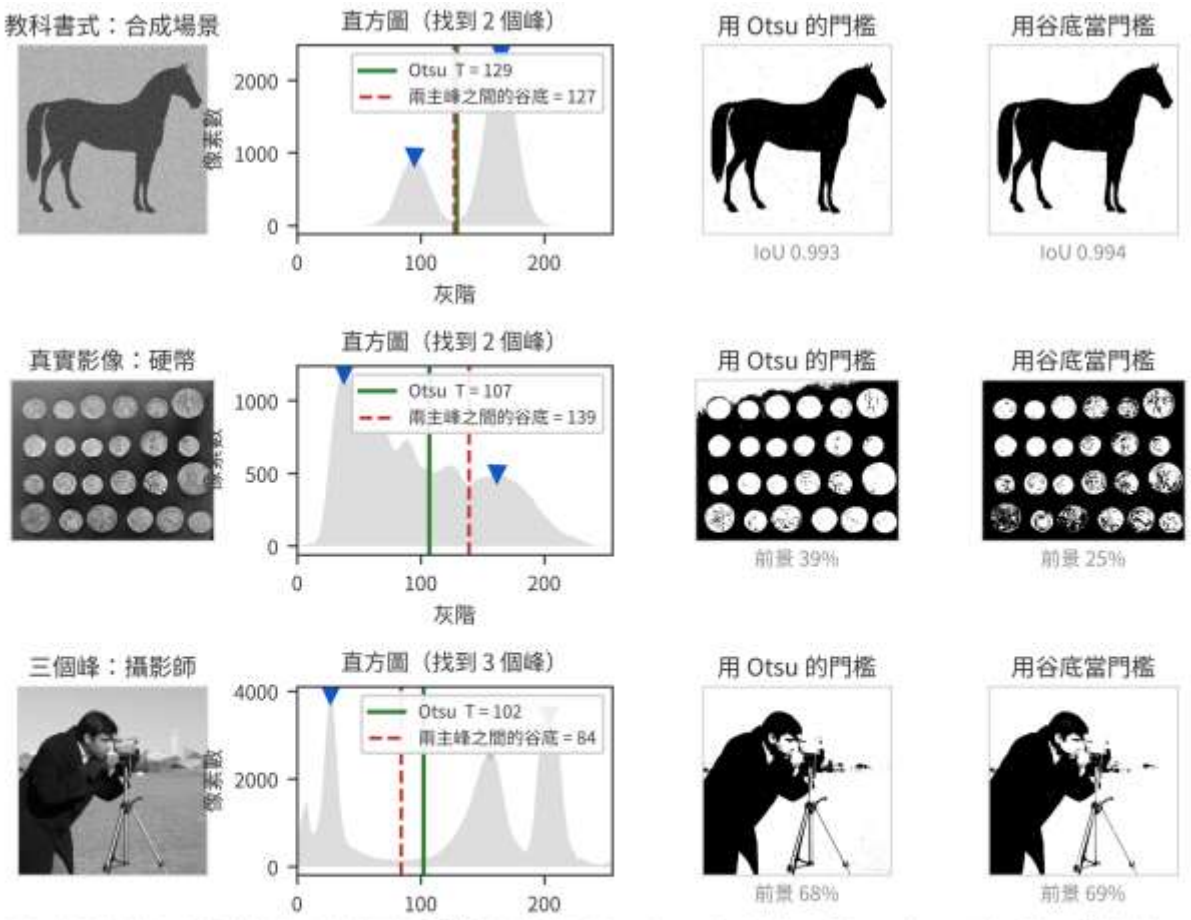
- 與第 6 章(頻率域)：邊緣是高頻、平坦區是低頻;Canny 內含「先低通平滑、再高通求梯度」的雙重濾波，正是第 6.6.4 節詳述的觀念。分割在本質上就是在區分影像中不同頻率特性的區域。
- 與第 7 章(影像還原)：還原品質直接影響分割——雜訊使邊緣偵測產生假邊、模糊使邊緣定位偏移。第 7.7 節已量化過這個連結。
- 與第 8 章(色彩處理)：彩色影像的分割常在 HSV 空間以顏色為依據(第 8.6 節的顏色分割)，或轉到 Lab 空間計算感知色差。
- 與第 10 章(二值影像處理)：門檻分割的輸出是二值影像，緊接著就用型態學清理雜點、連通元件標記來計數與量測——第 10 章是本章的直接下游。
- 與第 13 章(特徵擷取)：分割出的輪廓可計算形狀特徵(面積、周長、圓形度、Hu 矩)，供物件辨識使用。
- 與第 16 章(物件偵測與分割):YOLO 的實例分割、U-Net 的語意分割，是本章傳統方法的深度學習後繼者;9.6 節會說明兩者的分工與互補。
- 與第 17 章(邊緣部署)：傳統分割運算量小、不需訓練資料，在算力受限或樣本稀少的場合仍是首選——這是它在深度學習時代依然重要的原因。

9.2 門檻法：最簡單的分割

9.2.1 全域門檻

門檻法(Thresholding)是最直觀的分割：選一個灰階值 T 作為分界，大於 T 的像素歸為前景(設為白)、小於 T 的歸為背景(設為黑)。當前景與背景的灰階差異明顯、打光均勻時，一個固定的全域門檻就能漂亮地切開兩者。它的關鍵完全繫於 T 的選擇——選對了乾淨俐落，選錯了不是前景殘缺就是背景混入。

如何選 T?最理想的情況是影像的直方圖呈現雙峰(bimodal)：前景與背景各自形成一個峰，兩峰之間的谷底就是理想門檻。這也把我們帶回第 5 章——直方圖不只是診斷曝光的工具，更是選擇分割門檻的依據。

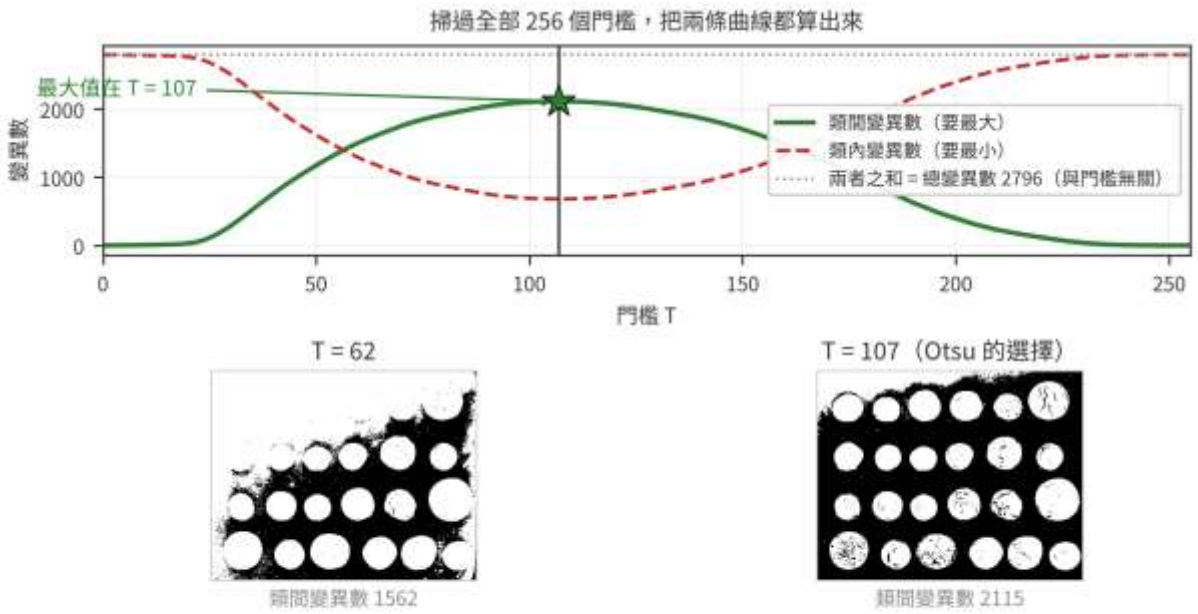


第一排是刻意造出來的理想情形，兩個峰正好落在 95 與 165 上，谷底 127 與 Otsu 的 129 只差 2 階——課本畫的就是這一排。第二排是真實影像：谷底在 139，Otsu 卻選 107，差了 32 階。因為 Otsu 最大化的是類間變異數，這個量會把每一類的像素數與分散程度一起算進去，不是單純找最低點。而且看右邊兩格就知道，Otsu 偏離谷底的方向是對的：Otsu 的門檻把硬幣完整留下，谷底那個門檻反而把硬幣表面的刻紋整片挖空。「Otsu 就是找谷底」只在兩類勢均力敵時才成立，而且谷底也未必是比較好的答案。第三排有 3 個峰，Otsu 仍然照給一個數字而且不會警告你——演算法永遠會回答，但答案不一定有意義。

圖 9-3　雙峰、谷底與 Otsu：課本畫的那一排只是三種情形之一

9.2.2 Otsu 自動門檻

人工看直方圖選門檻既主觀又難以自動化。日本學者大津展之(Nobuyuki Otsu)於 1979 年提出的方法，能自動找出最佳門檻：它遍歷所有可能的門檻值，對每一個門檻把像素分成前景與背景兩群，計算兩群的「類間變異數」(between-class variance)，選擇使類間變異數最大的那個門檻。直覺上，最好的門檻應該讓兩群「各自內部愈集中、彼此之間離得愈遠」——類間變異數最大，正對應這個目標。Otsu 法有一個漂亮的性質：最大化類間變異數，等價於最小化類內變異數，而前者的計算可以用遞推快速完成，因此效率極高。



Otsu 的全部內容就是這張圖：把 256 個門檻通通試一遍，挑類間變異數最大的那一個。本圖實算的結果是 T = 107，與 OpenCV 的 cv2.threshold 回傳值 107 完全一致。注意兩條曲線永遠加起來等於總變異數（灰色虛線），所以「類間最大」與「類內最小」是同一件事，不必分別計算——這個恆等式正是 Otsu 演算法能夠快速執行的原因。

圖 9-4　Otsu 到底在最大化什麼：把 256 個門檻通通算一遍

Otsu 法的前提是直方圖確實雙峰。若前景只佔極小面積(直方圖被背景的單峰主宰)、或前景背景灰階重疊嚴重，Otsu 就會失準——這時要嘛先做前處理拉開對比，要嘛改用下一節的自適應門檻。

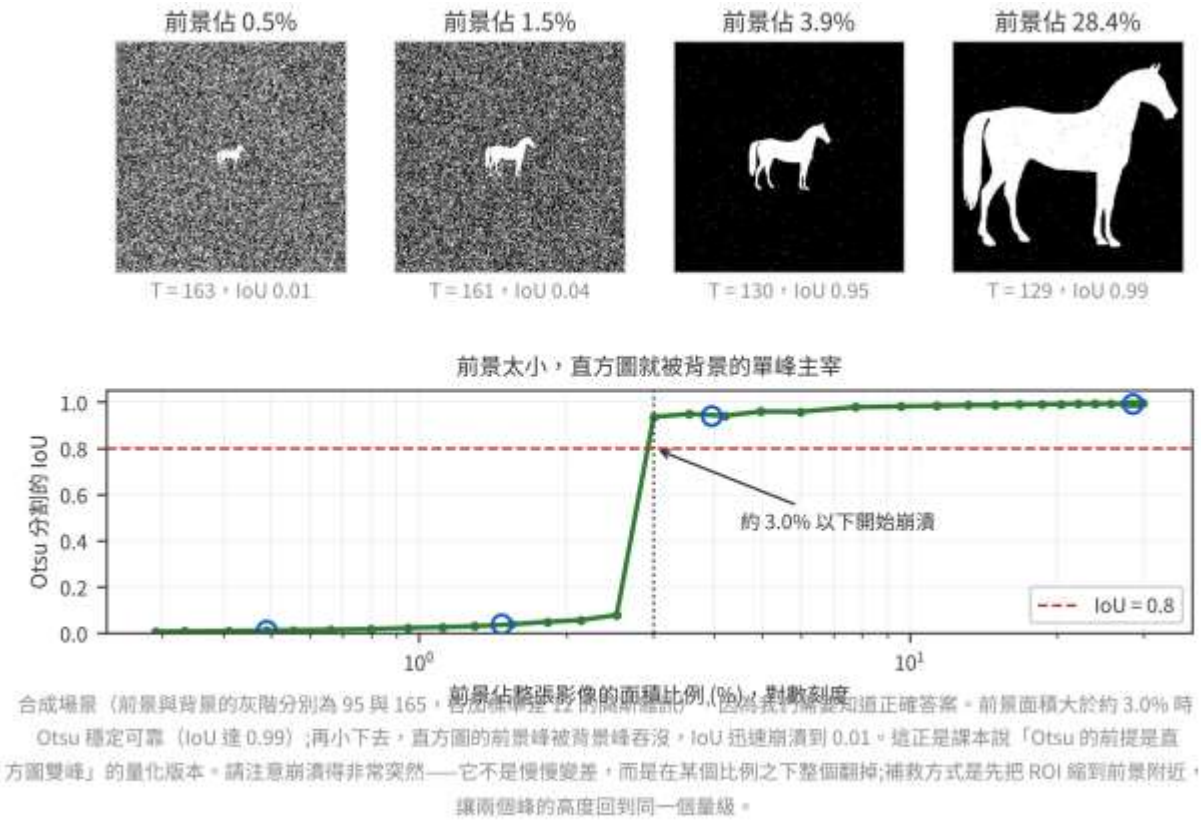
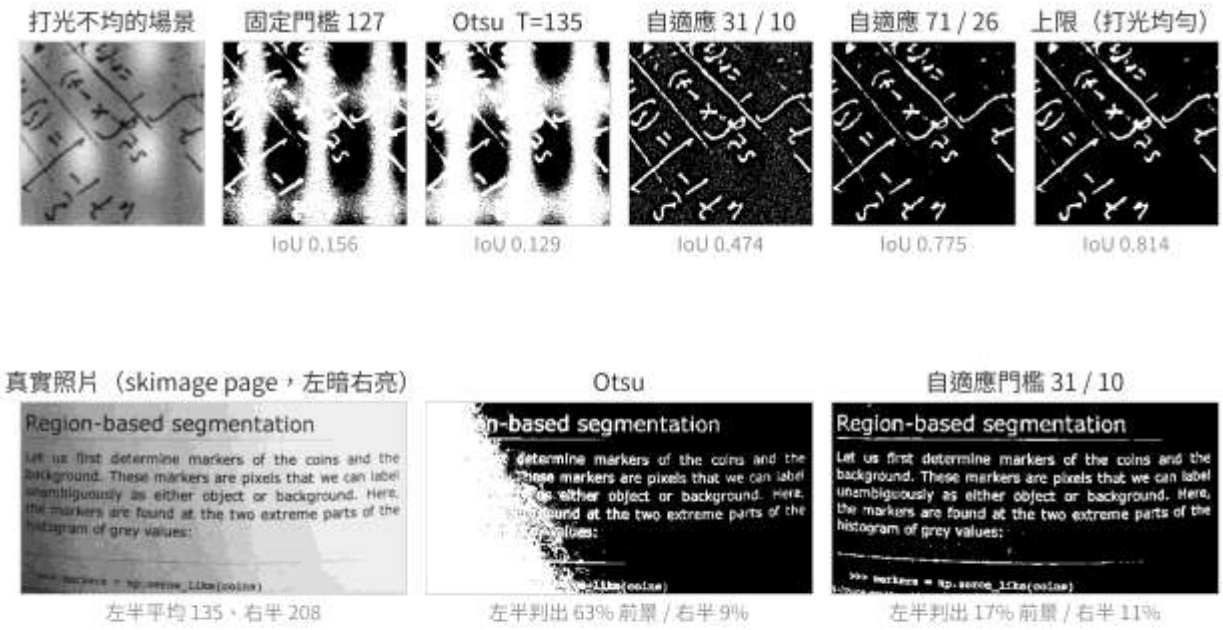


圖 9-5　前景小到什麼程度，Otsu 就會崩潰

9.2.3 自適應門檻

全域門檻有個致命弱點：它假設整張影像的最佳門檻都一樣。但真實場景常有打光不均——一張紙的左半邊在陽光下、右半邊在陰影中，任何單一門檻都會顧此失彼(這正是第 8 章「洋裝效應」在灰階世界的翻版)。自適應門檻(Adaptive Threshold)的解法是：不用一個全域門檻，而是為每個像素、依據它周圍鄰域的局部統計量(平均值或高斯加權平均)各自計算一個門檻。如此一來，亮區用高門檻、暗區用低門檻，打光不均的問題迎刃而解。文件掃描 App 把泛黃、有陰影的紙張轉成黑白分明的掃描檔，靠的就是自適應門檻。



上排是合成場景，所以可以算 IoU：固定門檻 0.16、Otsu 0.13、程式 9-1 預設參數的自適應門檻 0.47、調過參數的 0.77。請特別注意最後一格——就算把打光弄成完全均勻、再用事後挑的最佳門檻 143，IoU 也只有 0.81;剩下的損失來自鏡頭模糊與雜訊。跟門檻法一點關係都沒有。調過參數的自適應門檻已經拿到這個上限的 95%。先量出上限，才知道一個方法還有多少改善空間;否則很容易把物理上做不到的事怪到演算法頭上。下排是真實照片，沒有正確答案，改量左右半邊各判出多少前景：文字分布大致均勻，Otsu 卻判出 63% 對 9%，自適應門檻則是 17% 對 11%。

圖 9-6　一條水平線救不了一條斜坡——順便量出門檻法的上限

程式 9-1 三種門檻法

```
import cv2

gray = cv2.imread("document.jpg", cv2.IMREAD_GRAYSCALE)

# --- 全域固定門檻 ---
_, fixed = cv2.threshold(gray, 127, 255, cv2.THRESH_BINARY)

# --- Otsu 自動門檻(回傳自動選出的 T) ---
T, otsu = cv2.threshold(gray, 0, 255,
                        cv2.THRESH_BINARY + cv2.THRESH_OTSU)
print("Otsu 選出的門檻:", T)

# --- 對付雜訊:先高斯模糊再 Otsu(第 5 章前處理) ---
blur = cv2.GaussianBlur(gray, (5, 5), 0)
_, otsu2 = cv2.threshold(blur, 0, 255,
                         cv2.THRESH_BINARY + cv2.THRESH_OTSU)

# --- 自適應門檻:對付打光不均 ---
adap = cv2.adaptiveThreshold(gray, 255,
        cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
        cv2.THRESH_BINARY, blockSize=31, C=10)

cv2.imwrite("th_fixed.png", fixed)
cv2.imwrite("th_otsu.png", otsu)
cv2.imwrite("th_adaptive.png", adap)
```

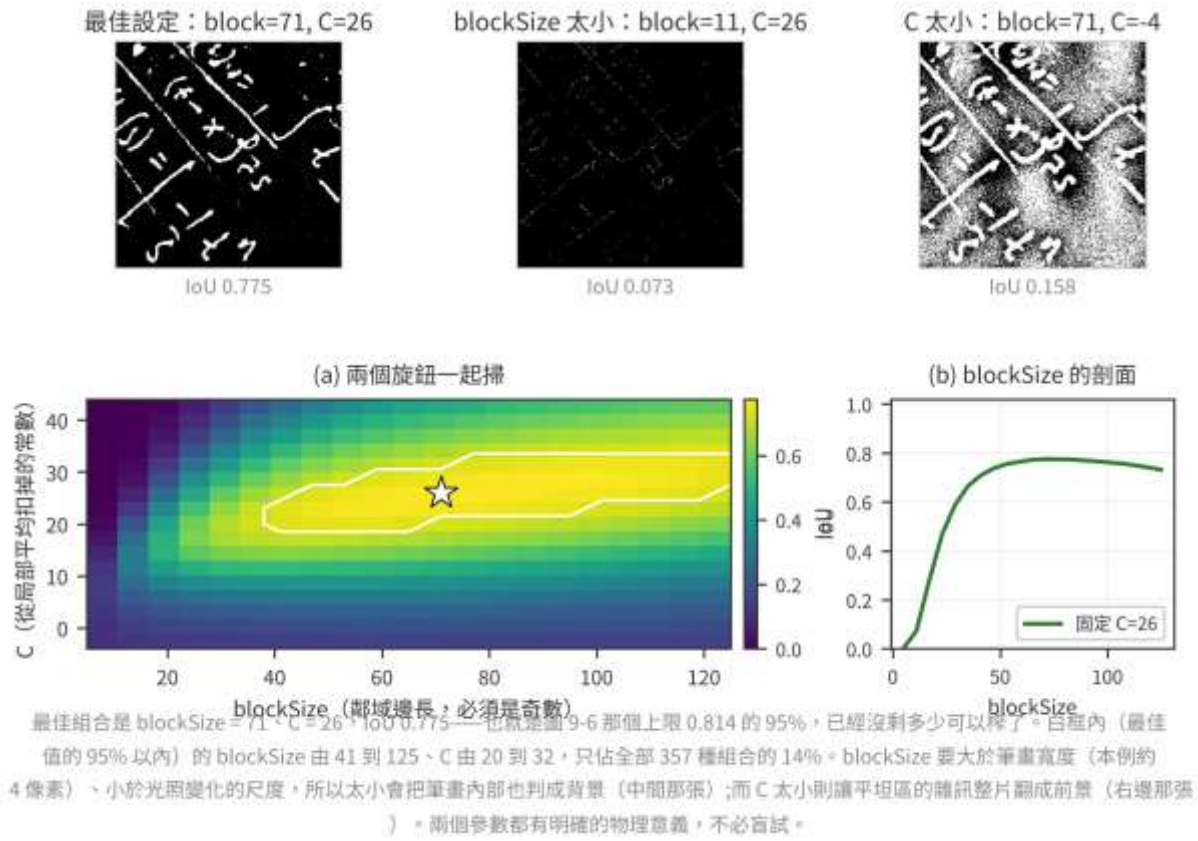


圖 9-7　自適應門檻的兩個旋鈕各有明確的物理意義

9.2.4 推導：Otsu 為什麼能在一瞬間試完 256 個門檻

9.2.2 節說 Otsu「遍歷所有可能的門檻值」。乍聽之下這是一件昂貴的事：每換一個門檻，就要把整張影像重新分成兩群、重算兩群的平均與變異數。一張一千萬像素的影像乘上 256 個門檻，那是二十幾億次運算。但 Otsu 實際上只要掃一次直方圖就算完了，而且與影像的大小完全無關。這一節說明為什麼。

第一個關鍵是：所有的計算都只需要直方圖。把灰階 i 的像素比例記為 p(i)。門檻取 T 時，背景群是灰階 0 到 T，前景群是 T+1 到 255，於是背景的比例 w0(T) = p(0) + p(1) + … + p(T)，背景的平均 m0(T) = [0·p(0) + 1·p(1) + … + T·p(T)] / w0(T)。請注意這兩個式子的分子都是「累積和」：w0(T) 是 p 的前綴和，m0 的分子是 i·p(i) 的前綴和。前綴和可以用遞推一次算完——w0(T) = w0(T−1) + p(T)，只要一次加法。前景那一群則由全體減去背景得到，w1(T) = 1 − w0(T)，而 m1(T) = [mT − Σ i·p(i)] / w1(T)，其中 mT 是整張影像的平均。所以掃過 256 個門檻的總成本是 256 次加法與

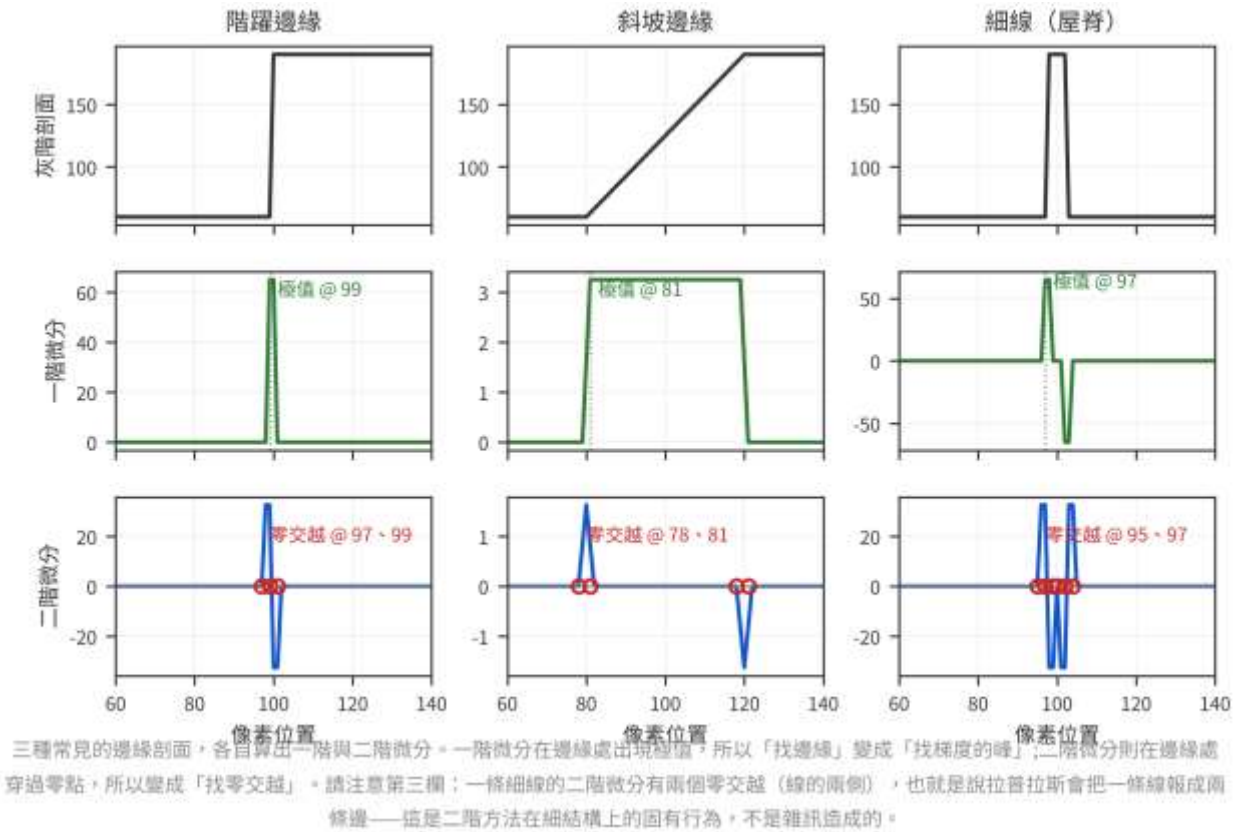
256 次除法，而建直方圖本身是 O(N)。這就是為什麼 cv2.threshold 加上 THRESH_OTSU 幾乎不花時間。

第二個關鍵是那個漂亮的恆等式。把整張影像的總變異數記為 σ^2，它可以被分解成兩項：$\sigma^2 = \sigma^2_{within}(T) + \sigma^2_{between}(T)$，其中 $\sigma^2_{between}(T) = w0 \cdot w1 \cdot (m0 - m1)^2$。圖 9-4 把這兩條曲線都實際畫了出來，它們的和確實是一條與門檻無關的水平線（該圖量到的總變異數是 2796）。這個恆等式的意義非常實用：既然兩者相加是常數，那麼「讓類內變異數最小」與「讓類間變異數最大」就是同一件事。而類間變異數只需要 w0、w1、m0、m1 四個已經算好的量，類內變異數卻要回頭掃每一個像素——Otsu 選了便宜的那一個。

第三件事是這個推導順便揭露的一個常見誤解。$\sigma^2_{between} = w0 \cdot w1 \cdot (m0 - m1)^2$ 這個式子裡，除了「兩群離得多遠」的 $(m0 - m1)^2$ 之外，還乘上了「兩群各佔多少」的 $w0 \cdot w1$。這一項在 $w0 = w1 = 0.5$ 時最大，也就是說 Otsu 內建一個偏好：它傾向把影像切成面積相當的兩半。圖 9-3 第二排量到的現象就是這麼來的——硬幣影像的直方圖谷底在 139，Otsu 卻選了 107，因為往下移可以讓兩群的面積更接近。所以「Otsu 就是自動找谷底」這句話並不正確;更精確的說法是「Otsu 在找一個能把影像切得最開、同時兩邊份量又不會太懸殊的門檻」。圖 9-5 的實驗則是這個偏好的反面：當前景只佔 3% 以下時，$w0 \cdot w1$ 這一項逼著門檻往背景那邊靠，分割就整個垮掉。

9.3 邊緣偵測

邊緣是影像中灰階劇烈變化之處，通常對應物體的輪廓、表面的轉折或深度的不連續。從第 6 章的頻率觀點看，邊緣就是高頻;而偵測高頻變化的數學工具，是微分。



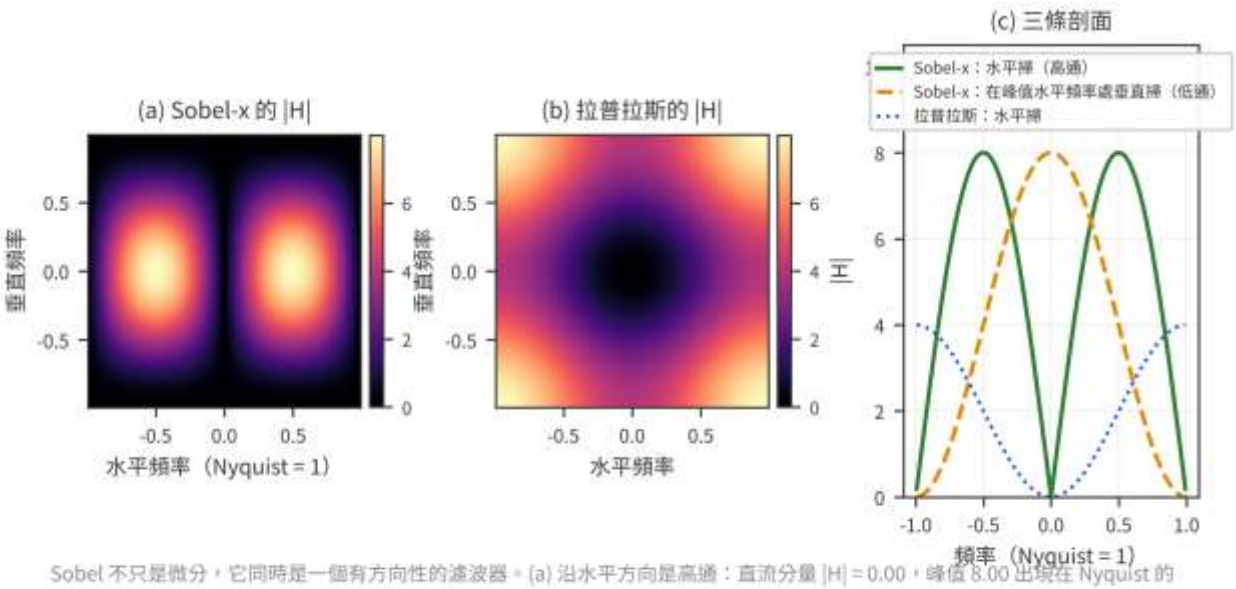
三種常見的邊緣剖面，各自算出一階與二階微分。一階微分在邊緣處出現極值，所以「找邊緣」變成「找梯度的峰」；二階微分則在邊緣處穿過零點，所以變成「找零交越」。請注意第三欄：一條細線的二階微分有兩個零交越（線的兩側），也就是說拉普拉斯會把一條線報成兩條邊——這是二階方法在細結構上的固有行為，不是雜訊造成的。

圖 9-8　一階微分找峰，二階微分找零交越

9.3.1 梯度與一階微分：Sobel

影像的梯度是一個向量，由水平方向與垂直方向的偏微分組成，其大小代表灰階變化的劇烈程度、方向代表變化最快的方向(垂直於邊緣走向)。在離散的數位影像上，微分以差分近似，並包裝成卷積核。最常用的是 Sobel 運算子：它在計算某方向差分的同時，對垂直方向做加權平均，兼具「求梯度」與「抗雜訊」兩個功能——這也呼應第 6 章的觀點：Sobel 是一個帶有平滑效果的方向性高通濾波器。取得水平梯度 Gx 與垂直梯度 Gy 後，梯度大小為兩者平方和開根號，對它做門檻化即可得到邊緣圖。

相關的運算子還有 Scharr(比 Sobel 更精確的梯度近似，適合小核)、Prewitt(較簡單的差分)。它們的共同侷限是：對雜訊敏感(微分會放大雜訊)，且邊緣往往偏粗、需要後續細化。



Sobel 不只是微分，它同時是一個有方向性的濾波器。(a) 沿水平方向是高通：直流分量 |H| = 0.00，峰值 8.00 出現在 Nyquist 的 0.50 倍處。沿垂直方向卻是低通——固定在同一個水平頻率，垂直頻率走到 Nyquist 的一半時響應由 8.00 掉到 4.05（剩 51%），走到 Nyquist 則恰好歸零。這不是巧合：Sobel 可分解為水平的 [−1, 0, 1] 與垂直的 [1, 2, 1]，而 [1, 2, 1] 的頻率響應 2 + 2 cos(ω) 在 ω = π 處正是 0。這正是 9.3.1 節說「Sobel 在求差分的同時對垂直方向做加權平均」的頻率域說法，也是它比單純的差分抗雜訊的原因。(b) 拉普拉斯是各向同性的高通，沒有平滑，所以更怕雜訊。

圖 9-9　Sobel 是一個有方向性的濾波器：一邊高通、一邊低通

9.3.2 二階微分：Laplacian 與 LoG

一階微分在邊緣處產生極值，二階微分則在邊緣處產生零交越(zero-crossing)——由正轉負的那一點，正是邊緣的精確位置。拉普拉斯運算子(Laplacian)是二階微分的離散近似，定位精準且各向同性(不分方向)，但對雜訊極度敏感(第 5、6 章都提過，二階微分把雜訊放大得比一階更兇)。

解法是先平滑再微分：把高斯平滑與拉普拉斯結合，就是高斯拉普拉斯(Laplacian of Gaussian,LoG)，又因其剖面形似而俗稱「墨西哥帽」。LoG 先用高斯壓制雜訊(低通)，再用拉普拉斯找零交越(高通)——又是一個「低通加高通」的組合，與第 6 章的敘述完全一致。LoG 也是第 13 章 SIFT 特徵偵測的理論前身(SIFT 用高斯差 DoG 來近似 LoG)。

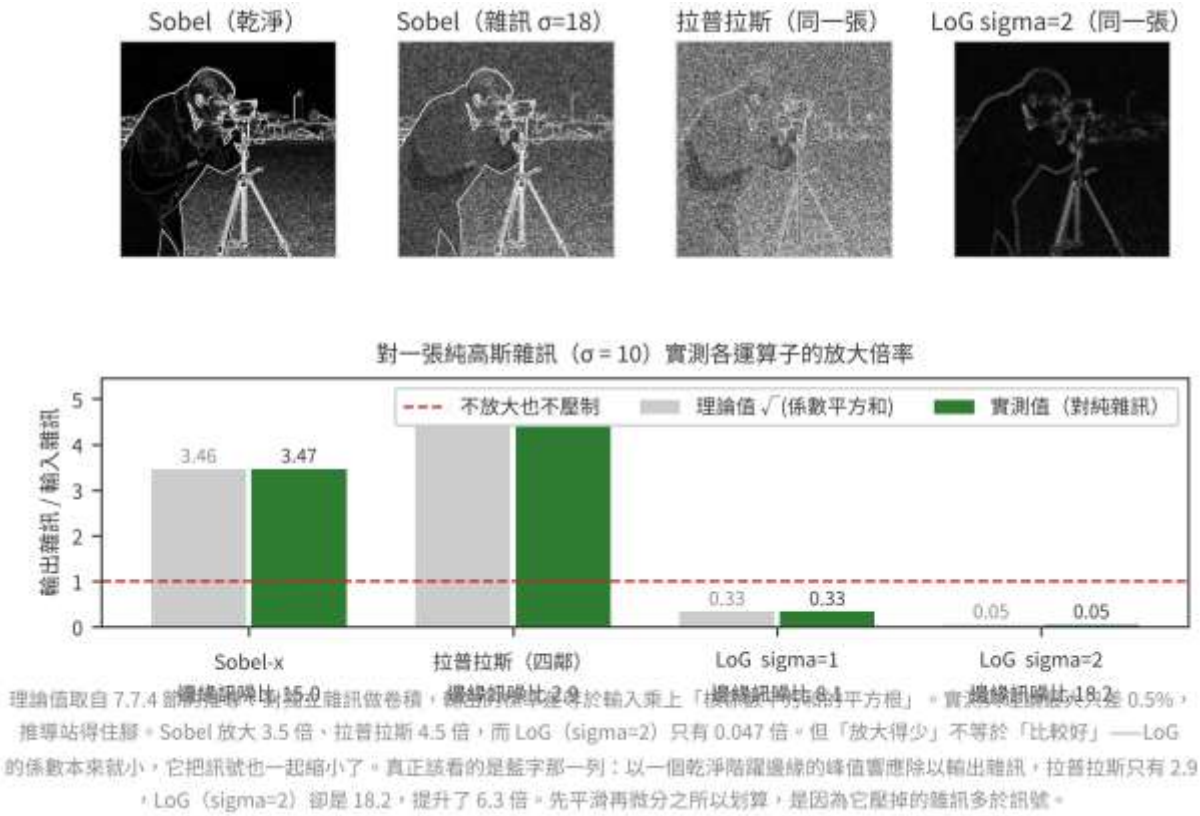


圖 9-10　微分把雜訊放大了多少：理論值與實測值對照

9.3.3 Canny：邊緣偵測的黃金標準

1986 年 John Canny 提出的邊緣偵測器，以「好的偵測、好的定位、單一響應」三個準則為設計目標，至今仍是最廣泛使用的邊緣偵測方法。它是一條精心設計的四步驟管線，每一步都呼應本書前面的觀念：

3. 高斯平滑：先用高斯濾波器壓制雜訊(第 5 章低通)，避免後續微分放大雜訊。sigma 的大小決定了對雜訊的容忍度與細節的保留度，是最關鍵的參數。
4. 計算梯度：以 Sobel 求出每個像素的梯度大小與方向(第 9.3.1 節)。
5. 非極大值抑制(Non-Maximum Suppression)：沿著梯度方向，只保留局部最大的像素，把粗邊緣細化成單像素寬的線。這是 Canny 邊緣俐落的關鍵。
6. 雙門檻與遲滯連接(Hysteresis)：設高、低兩個門檻。高於高門檻的必定是邊緣(強邊);低於低門檻的必定不是;介於兩者之間的(弱邊)，只有在「連接到強邊」

時才保留。這個巧妙的設計，既能避免雜訊造成的假邊，又能保住真實但較弱的邊緣不致斷裂。

Canny 的成功，正是「把多個簡單觀念(平滑、微分、抑制、連接)精心組合成一條管線」的典範——這也是叫獸開講「觀點翻轉」之外，工程上另一種創造價值的方式：巧妙的組合。

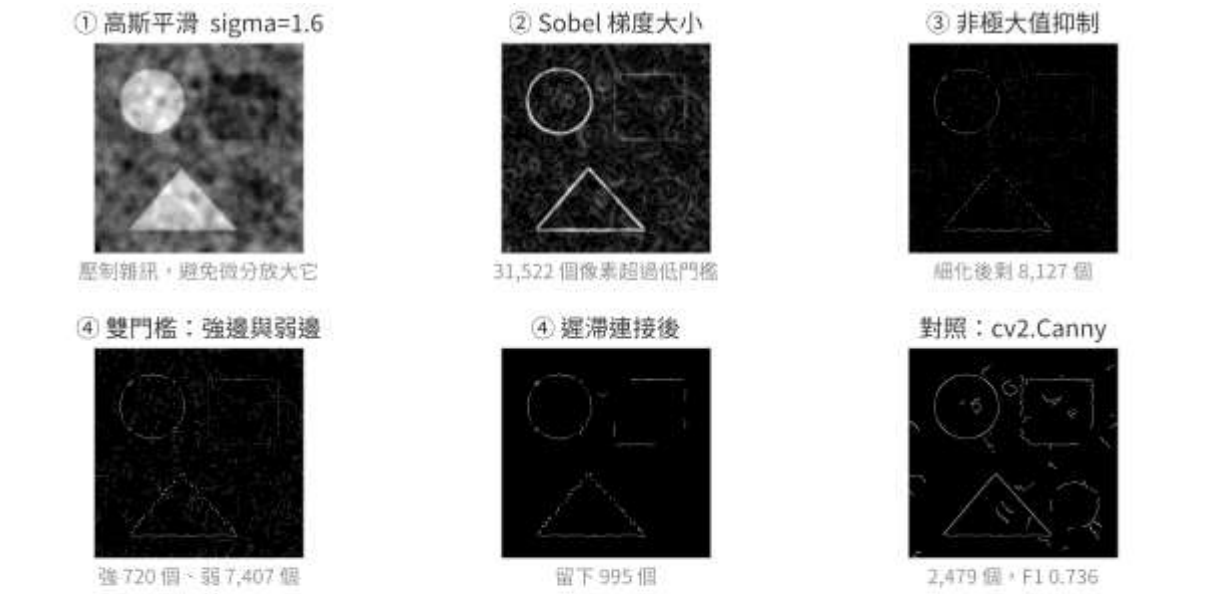


圖 9-11　Canny 的四個步驟，每一步的像素數都在上面

程式 9-2 Sobel、LoG 與 Canny 邊緣偵測

```
import cv2, numpy as np

gray = cv2.imread("parts.jpg", cv2.IMREAD_GRAYSCALE)

# --- Sobel 一階梯度 ---
gx = cv2.Sobel(gray, cv2.CV_64F, 1, 0, ksize=3)
gy = cv2.Sobel(gray, cv2.CV_64F, 0, 1, ksize=3)
mag = cv2.magnitude(gx, gy)
sobel = cv2.convertScaleAbs(mag)

# --- Laplacian of Gaussian(先平滑再二階微分) ---
blur = cv2.GaussianBlur(gray, (3, 3), 0)
log = cv2.Laplacian(blur, cv2.CV_64F, ksize=3)
log = cv2.convertScaleAbs(log)

# --- Canny(黃金標準) ---
edges = cv2.Canny(gray, threshold1=80, threshold2=160)
```

```
# 自動決定門檻:以中位數為基準(實務常用技巧)
v = np.median(gray)
lo = int(max(0, 0.66 * v))
hi = int(min(255, 1.33 * v))
edges_auto = cv2.Canny(gray, lo, hi)

cv2.imwrite("edge_sobel.png", sobel)
cv2.imwrite("edge_log.png", log)
cv2.imwrite("edge_canny.png", edges)
```

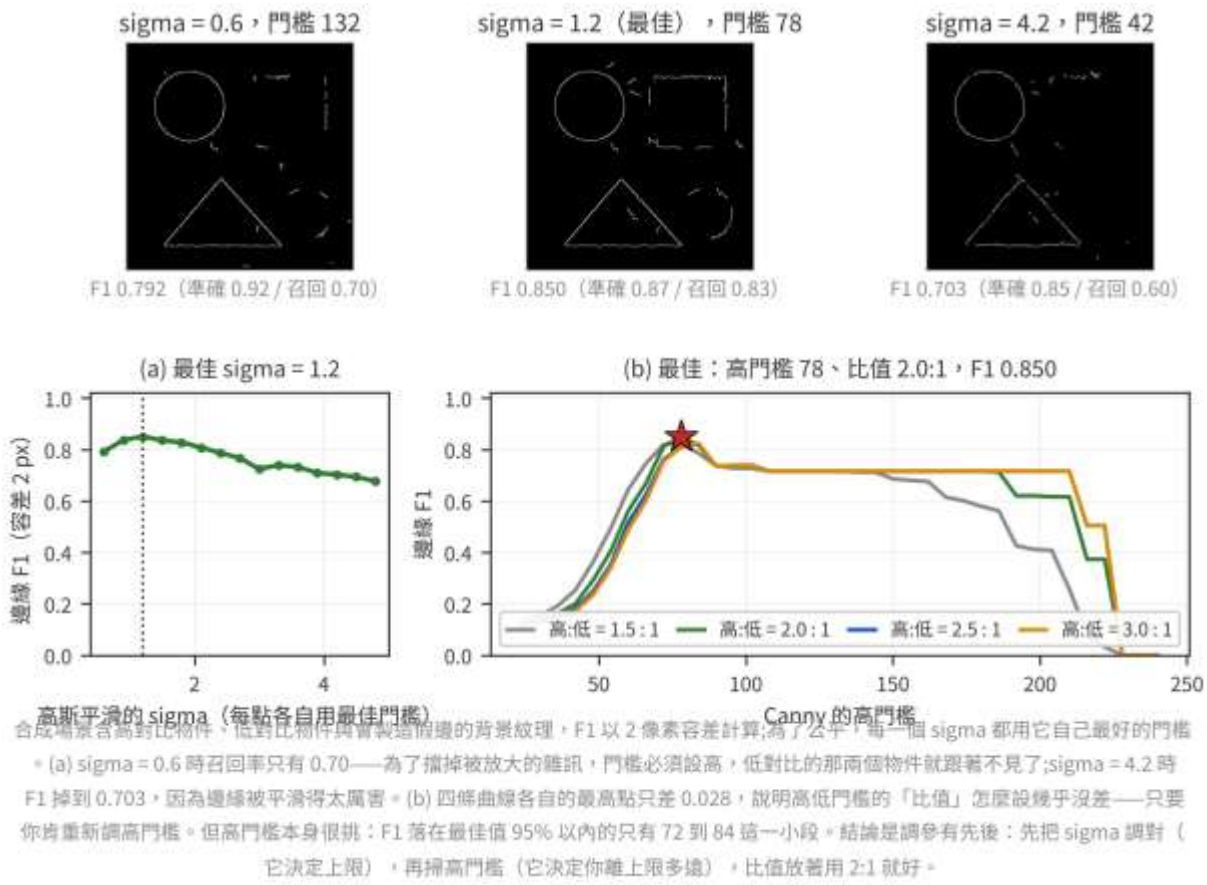


圖 9-12　Canny 的三個旋鈕：sigma 決定上限，門檻決定你離上限多遠

9.3.4 推導：非極大值抑制為什麼需要梯度方向

9.3.3 節把非極大值抑制描述成「沿著梯度方向，只保留局部最大的像素」。很多人讀過就跳過去了，但這一步其實是 Canny 四個步驟裡最精巧的一個，而且「為什麼非得沿梯度方向不可」值得算清楚。

先看問題出在哪裡。一條真實的邊緣經過高斯平滑之後，不會是一條一像素寬的線，而是一道有寬度的斜坡。對寬度為 w 的斜坡求梯度，得到的是一個寬度也大約是 w 的隆起——若直接用門檻切，一條邊緣會變成好幾個像素寬的粗帶子，而且寬度還

隨門檻高低而變。這對後續的霍夫轉換（每個像素投一票）與輪廓追蹤都是災難。我們要的是「把這道隆起壓成它的脊線」。

關鍵的幾何事實是：梯度向量永遠垂直於邊緣的走向。這可以直接從定義看出來——沿著邊緣走，灰階不變，所以方向導數為零;而梯度與某方向的內積就是該方向的方向導數，因此梯度必定垂直於邊緣。於是「跨過邊緣」的方向就是梯度方向，而隆起的剖面正好出現在這個方向上。非極大值抑制做的事就是：在每個像素處沿著梯度方向往前後各看一格，若自己不是三者中最大的就歸零。剩下的那些點，正是隆起的脊。

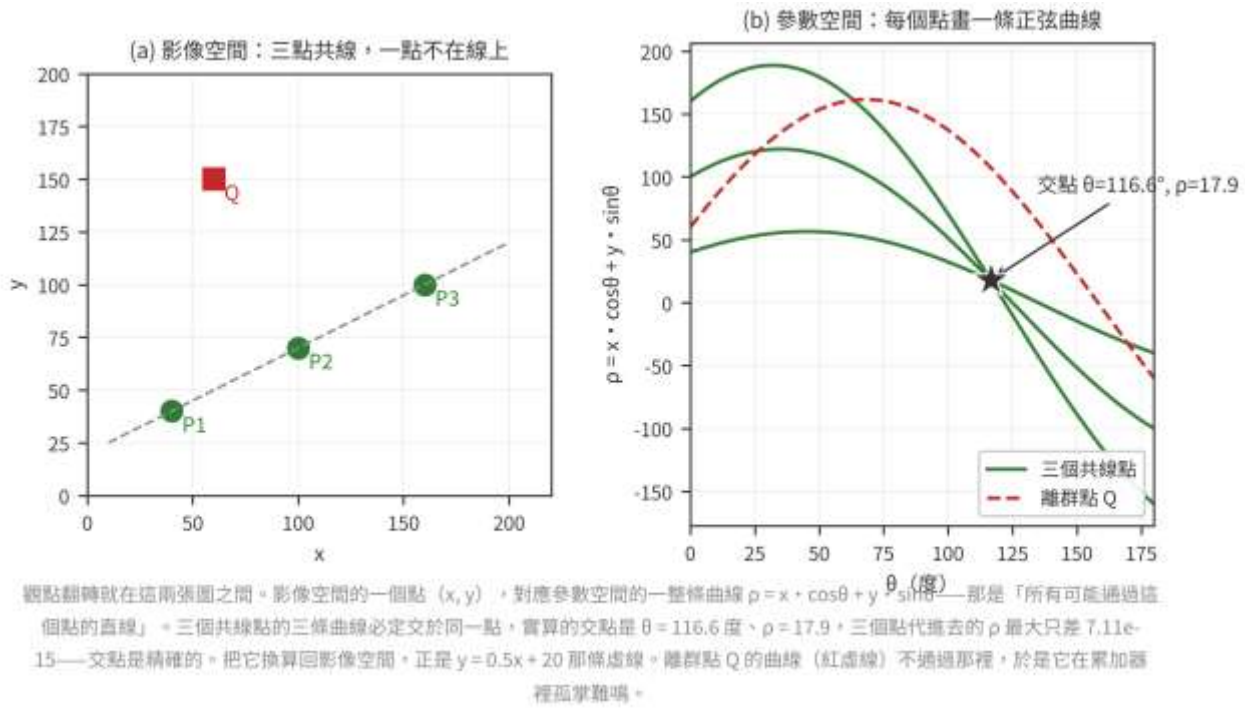
方向為什麼要量化成四個？因為離散影像上「往前走一格」只有八個鄰居，對應四條軸線：0 度（水平）、45 度、90 度、135 度。實作上把梯度角度除以 45 再四捨五入取模 4，就決定了要跟哪兩個鄰居比。這個量化會帶來最多 22.5 度的方向誤差，代價是邊緣位置可能偏半個像素;想更精準的話可以沿梯度方向做線性內插，取出兩個次像素位置的梯度值再比較，這也是多數高品質實作的做法。

效果有多大，可以直接量。圖 9-11 在同一張合成影像上實測：梯度圖中超過低門檻的像素有 31,522 個，經過非極大值抑制之後只剩 8,127 個，一口氣少掉 74%，而邊緣的位置完全沒有移動。換句話說，這一步丟掉的全是「同一條邊緣的重複報告」。也正因為輸出是單像素寬，下一步的雙門檻與遲滯連接才能用「連通性」來判斷弱邊的去留——若邊緣是粗的，連通性就失去意義了。四個步驟環環相扣，少一個都不行。

9.4 霍夫轉換：參數空間的投票

9.4.1 直線偵測的核心思想

邊緣偵測告訴我們「哪些像素在邊緣上」，但它給的是一堆散亂的邊緣點，並沒有告訴我們「這些點排成了一條直線」。霍夫轉換(Hough Transform)接手這個工作，而它的思想正是叫獸開講中那次精彩的觀點翻轉。



觀點翻轉就在這兩張圖之間。影像空間的一個點 (x, y)，對應參數空間的一整條曲線 ρ = x·cosθ + y·sinθ——那是「所有可能通過這個點的直線」。三個共線點的三條曲線必定交於同一點，實算的交點是 θ = 116.6 度、ρ = 17.9，三個點代進去的 ρ 最大只差 7.11e-15——交點是精確的。把它換算回影像空間，正是 y = 0.5x + 20 那條虛線。離群點 Q 的曲線（紅虛線）不通過那裡，於是它在累加器裡孤掌難鳴。

圖 9-13　霍夫轉換的觀點翻轉：一個點變成一條正弦曲線

關鍵在於視角互換。在影像空間中，一條直線由通過它的所有點定義;霍夫轉換反過來問：對於影像中的每一個邊緣點，有哪些直線可能通過它?答案是無限多條，這些直線的參數在「參數空間」中構成一條曲線。於是每個邊緣點都在參數空間投下一條曲線(等於為所有可能通過它的直線各投一票)。如果影像中真的有一條直線，那麼這條線上的所有點，投出的曲線都會通過參數空間中同一個點——那個點的參數，正是這條直線的參數。於是「找直線」變成「在參數空間找得票最高的點」。

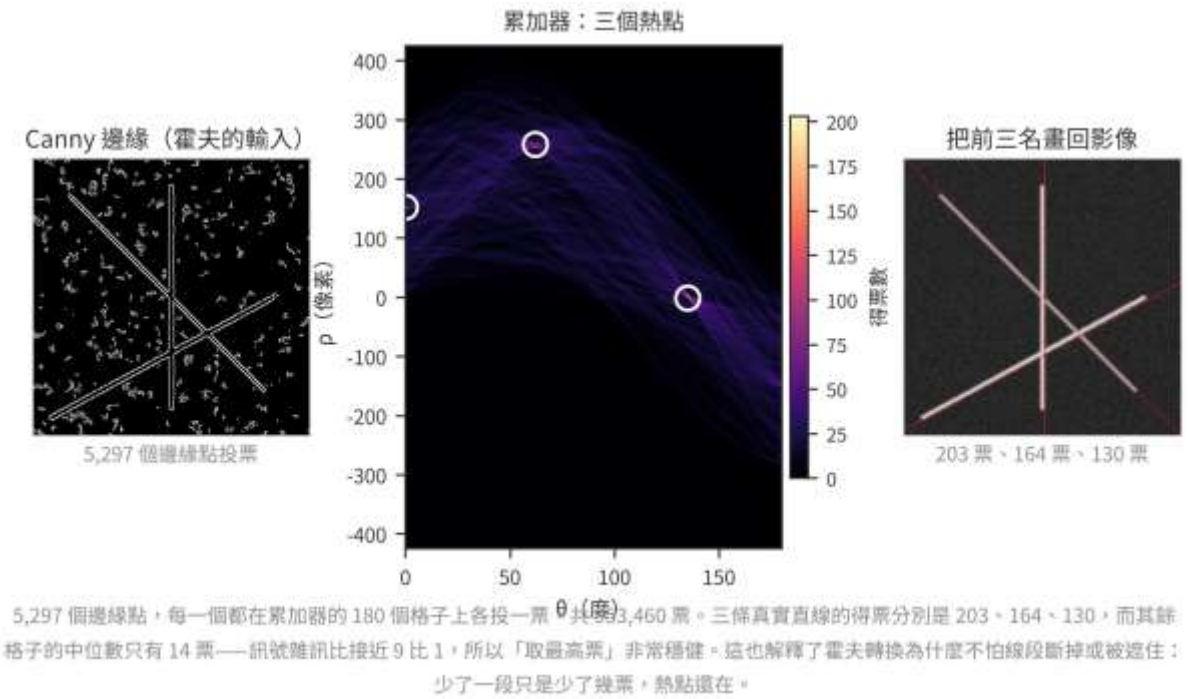
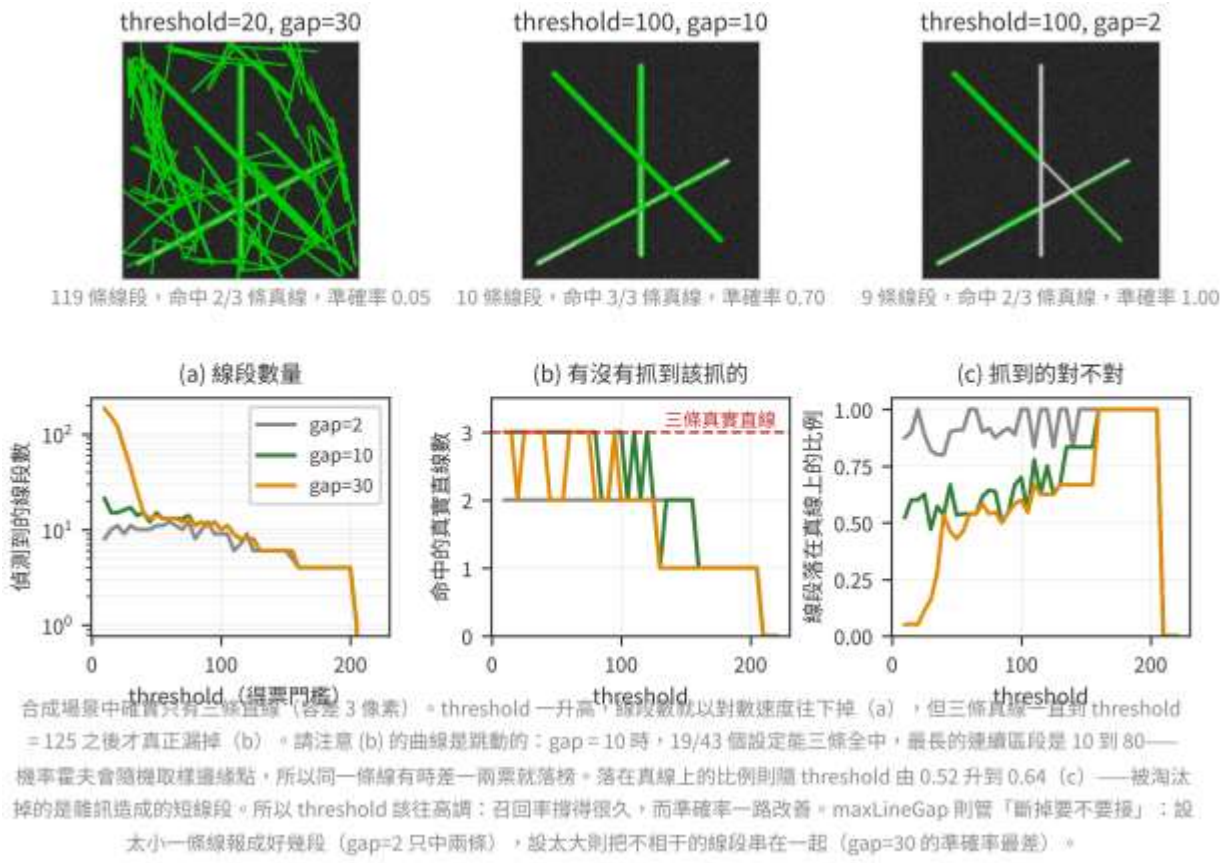


圖 9-14　累加器裡的三個熱點，以及它們的票數

實務上不用斜截式 y = mx + b，因為垂直線的斜率是無限大無法表示。改用 Duda 與 Hart 在 1972 年引入的極座標式：ρ = x·cosθ + y·sinθ，其中 ρ 是原點到直線的垂直距離、θ 是該垂線的角度。每個邊緣點在 (ρ, θ) 參數空間中對應一條正弦曲線，共線的點其正弦曲線交會於一點。參數空間被離散化成一個個格子(稱為累加器，accumulator)，每條曲線經過的格子就加一票，最後票數超過門檻的格子就是偵測到的直線。



合成場景中確實只有三條直線（容差 3 像素）。threshold 一升高，線段數就以對數速度往下掉（a），但三條真線一直到 threshold = 125 之後才真正漏掉（b）。請注意 (b) 的曲線是跳動的：gap = 10 時，19/43 個設定能三條全中，最長的連續區段是 10 到 80——機率霍夫會隨機取樣邊緣點，所以同一條線有時差一兩票就落榜。落在真線上的比例則隨 threshold 由 0.52 升到 0.64（c）——被淘汰掉的是雜訊造成的短線段。所以 threshold 該往高調：召回率撐得很久，而準確率一路改善。maxLineGap 則管「斷掉要不要接」：設太小一條線報成好幾段（gap=2 只中兩條），設太大則把不相干的線段串在一起（gap=30 的準確率最差）。

圖 9-15　機率霍夫的三個旋鈕各管什麼

9.4.2 圓偵測與霍夫梯度法

同樣的投票思想可以推廣到任何能用少數參數描述的形狀。圓需要三個參數：圓心 (a, b) 與半徑 r，因此累加器是三維的。若直接對三維空間投票，運算與記憶體成本都很高。

OpenCV 採用的霍夫梯度法(Hough Gradient Method)是一個聰明的加速：它利用一個幾何事實——圓上每一點的梯度方向都指向(或背離)圓心。因此不必盲目地為所有可能的圓心投票，而是讓每個邊緣點只沿著自己的梯度方向投票，票數自然集中到真正的圓心;先在二維空間定出圓心，再回頭決定半徑。這把三維投票拆解成兩個較低維的步驟，大幅提升效率。它廣泛用於偵測圓形物件——瞳孔、硬幣、球、儀表指針的軸心、工件上的圓孔。

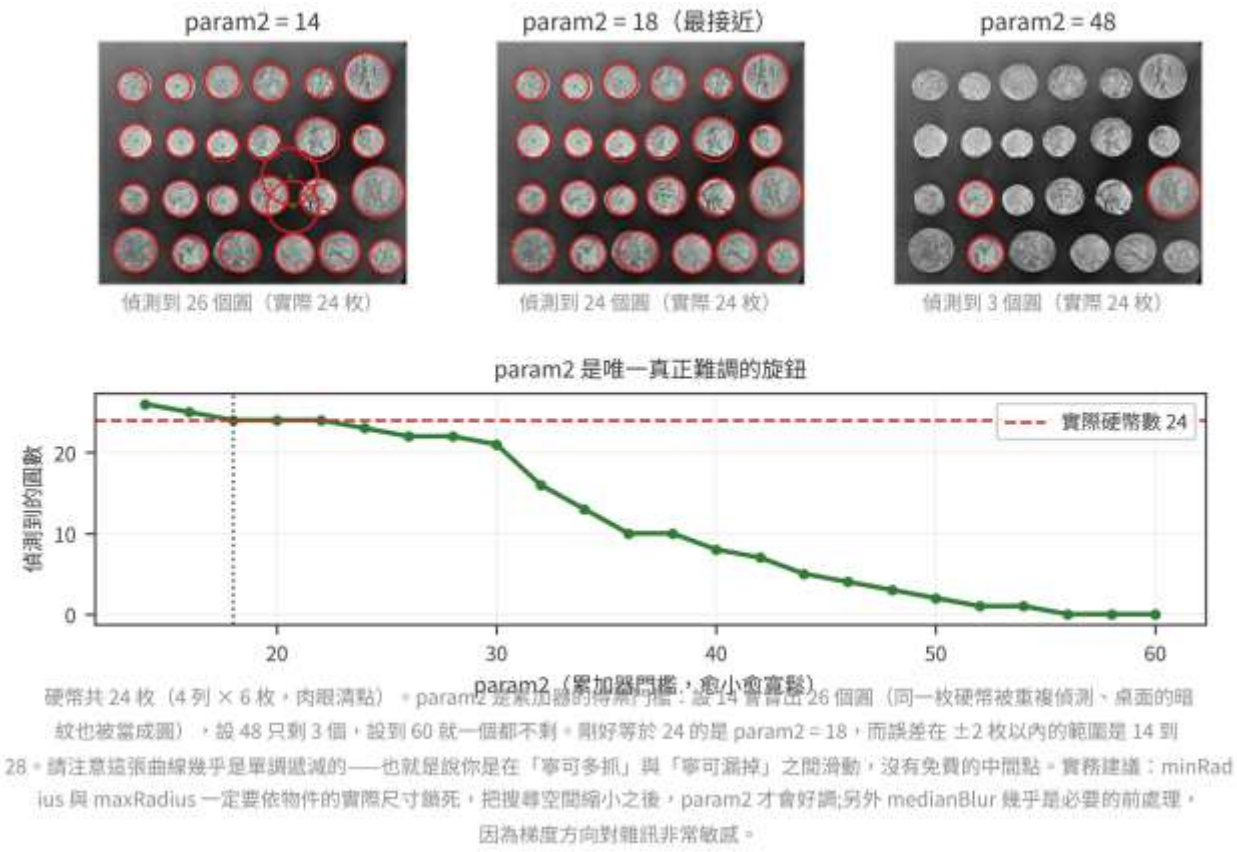


圖 9-16　param2 是霍夫圓偵測唯一真正難調的旋鈕

程式 9-3 霍夫直線與圓偵測

```
import cv2, numpy as np

gray = cv2.imread("road.jpg", cv2.IMREAD_GRAYSCALE)
edges = cv2.Canny(gray, 80, 160)                # 霍夫的輸入是邊緣圖

# --- 標準霍夫直線(回傳 rho, theta) ---
lines = cv2.HoughLines(edges, rho=1, theta=np.pi/180, threshold=150)

# --- 機率霍夫直線(回傳線段端點,實務更常用) ---
segments = cv2.HoughLinesP(edges, 1, np.pi/180, threshold=50,
                           minLineLength=50, maxLineGap=10)
img = cv2.cvtColor(gray, cv2.COLOR_GRAY2BGR)
if segments is not None:
    for x1, y1, x2, y2 in segments[:, 0]:
        cv2.line(img, (x1, y1), (x2, y2), (0, 255, 0), 2)

# --- 霍夫梯度法偵測圓 ---
coins = cv2.imread("coins.jpg", cv2.IMREAD_GRAYSCALE)
coins = cv2.medianBlur(coins, 5)                # 先去雜訊很重要
circles = cv2.HoughCircles(coins, cv2.HOUGH_GRADIENT, dp=1,
            minDist=30, param1=100, param2=30,
            minRadius=10, maxRadius=80)
if circles is not None:
    for a, b, r in np.uint16(np.around(circles))[0]:
        cv2.circle(img, (a, b), r, (0, 0, 255), 2)
```

```
cv2.imwrite("hough_lines.png", img)
```

9.4.3 推導：累加器要多大，投票要花多少錢

霍夫轉換的思想很美，但它是一個會吃記憶體的方法。9.4.2 節說「若直接對三維空間投票，運算與記憶體成本都很高」，這一節把那個「高」算成具體的數字，你才知道什麼時候可以用、什麼時候要換方法。

先算直線。極座標式 ρ = x·cosθ + y·sinθ 裡，θ 的範圍是 0 到 180 度，ρ 的範圍則是從 −D 到 +D，其中 D 是影像的對角線長度。若 θ 取 1 度一格、ρ 取 1 像素一格，累加器就是 2D × 180 格。以圖 9-14 的 300×300 場景為例，D = 425，累加器是 850 × 180 = 153,000 格，用 int32 存不到 0.6 MB，完全不是問題。就算換成 1080p 的畫面，D = 2203，累加器 4406 × 180 = 793,080 格，也只有 3 MB。投票的成本則是「邊緣點數 × θ 的格數」：圖 9-14 量到 5,297 個邊緣點，乘上 180 就是 953,460 次累加——這在現代處理器上是毫秒等級。

再算圓，差別立刻出現。圓需要三個參數（圓心 a、b 與半徑 r），累加器是三維的 H × W × R。以 coins 影像（303 × 384）為例，若半徑限定在 12 到 36 之間（R = 25），累加器是 303 × 384 × 25 = 2,908,800 格，已經是直線的 16 倍。若不限定半徑，讓 r 從 1 跑到 192，就是 22,339,584 格、89 MB，而且每個邊緣點要投的票也從 180 票暴增到數萬票。這就是為什麼 cv2.HoughCircles 的 minRadius 與 maxRadius 不是「可選參數」而是「必填參數」——不鎖住半徑，這個方法在實務上根本跑不動。

霍夫梯度法的加速就建立在一個幾何事實上：圓上任何一點的梯度方向，必定通過圓心。因此不必讓每個邊緣點對所有可能的 (a, b, r) 投票，只要沿著自己的梯度方向畫一條射線，對線上的點投票即可。這把三維的投票降成二維（先只找圓心），圓心確定之後再用「到圓心的距離直方圖」決定半徑。代價是它對梯度方向的準確度非常敏感——這也是為什麼程式 9-3 在偵測圓之前一定要先做 medianBlur：梯度方向被雜訊擾動幾度，射線就射偏了，票就投到別處去。圖 9-16 的實驗顯示，即使前處理做

對了，param2（累加器門檻）仍然是唯一難調的旋鈕：在 24 枚硬幣的影像上，誤差 ±2 枚以內的 param2 只有 14 到 28 這一小段。

最後是一個推廣的原則。霍夫轉換能處理任何「可以用少數參數描述」的形狀，但成本隨參數個數指數成長：直線兩個參數、圓三個、橢圓五個。橢圓的累加器在實務上已經大到不可行，所以現實中偵測橢圓多半改用「先找輪廓、再最小平方擬合」（第 13 章的 cv2.fitEllipse），而不是硬投票。判斷原則因此很清楚：參數少於三個時霍夫很好用，三個時要想辦法降維（像霍夫梯度法那樣），超過三個就該換一條路了。

9.5 基於區域的分割

9.5.1 區域成長

區域成長(Region Growing)體現了「基於相似性」的策略。它從一個或多個種子點出發，檢查種子的鄰居：若鄰居與種子(或目前區域)的性質夠相似(灰階差在容許範圍內)，就把鄰居併入區域，再從新併入的像素繼續向外檢查，像墨水在紙上暈開一樣，直到再也沒有相似的鄰居可併入。它的優點是能保證區域的連通性、對定義明確的區域效果好;缺點是結果高度依賴種子的選擇與相似度門檻，且對雜訊敏感。

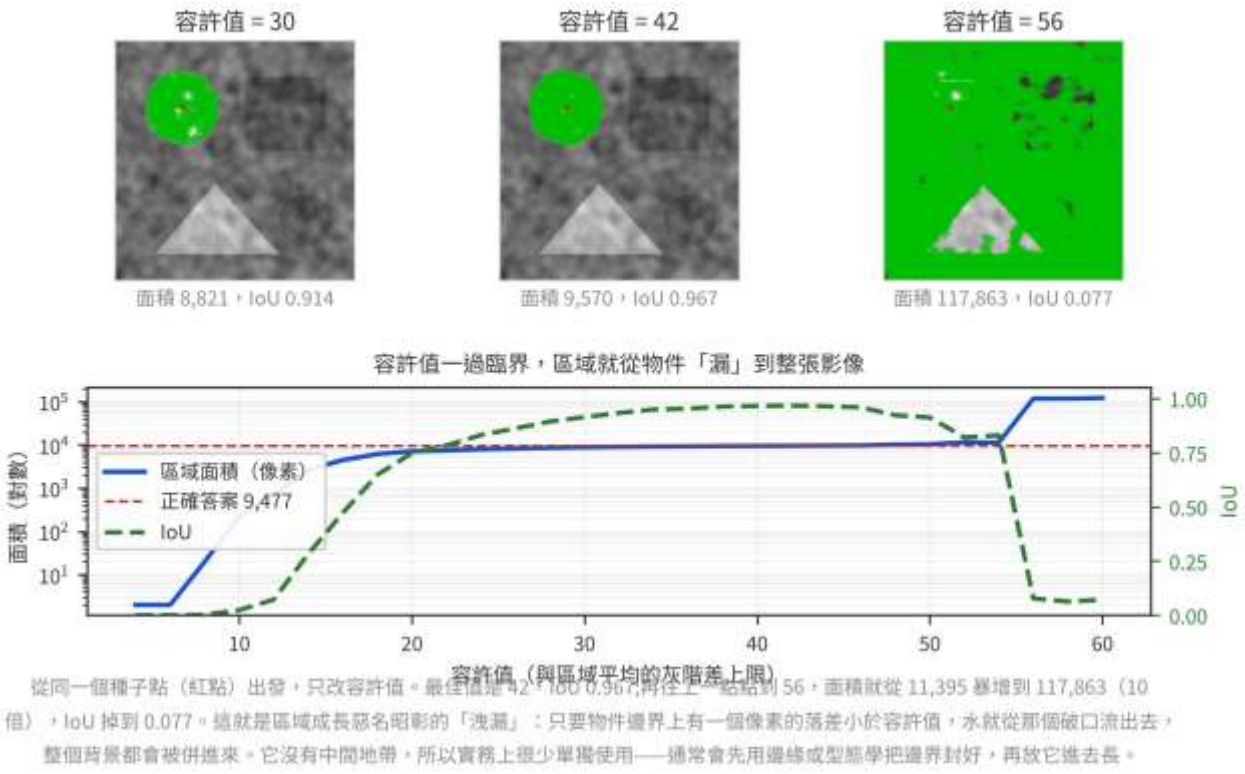
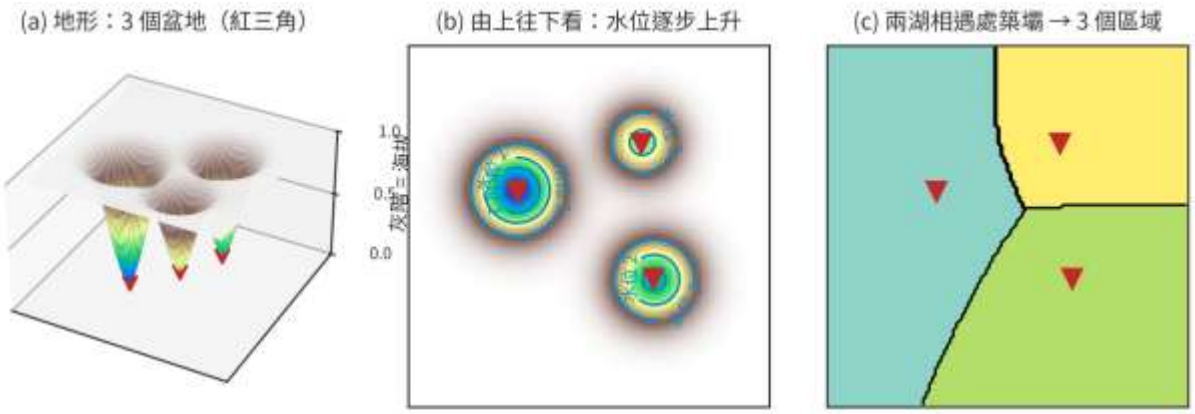


圖 9-17　區域成長的洩漏：容許值差一點，面積差十倍

9.5.2 分水嶺法

分水嶺法(Watershed)有一個極為優美的比喻。把灰階影像想像成一片地形：灰階值就是海拔，亮處是山峰、暗處是山谷(通常對梯度影像操作，則邊緣是山脊、區域內部是盆地)。現在想像從每個盆地的最低點開始注水，水位緩緩上升，不同盆地的水各自成湖;當兩個湖的水即將相遇時，在交界處築起一道堤壩——這些堤壩，就是分割的邊界。「分水嶺」之名由此而來。



把灰階值當成海拔：暗處是盆地，亮處是山脊。從每個局部極小值開始注水。水位慢慢升高;當兩個湖快要相連時，就在交界處築一道壩，那道壩就是分割邊界。本圖的地形有 3 個局部極小值，分水嶺就得到 3 個區域——這正是問題所在：區域數完全由局部極小值的個數決定，而真實影像的梯度圖上，雜訊會製造成千上萬個局部極小值。下一張圖會量給你看那個數字有多誇張。

圖 9-18　把灰階當地形：注水、相遇、築壩

分水嶺法天生擅長分開「相互接觸、黏在一起」的物件——這是門檻法最頭痛的情況(兩個相黏的細胞或零件，門檻分割會把它們當成一個)。但它有一個惡名昭彰的問題：過度分割(over-segmentation)。因為影像中每一個微小的局部極小值(常由雜訊造成)都會形成一個獨立的盆地，結果把影像切得支離破碎。

解法是標記控制的分水嶺(Marker-Controlled Watershed)：不讓水從所有局部極小值自由湧出，而是由使用者(或前處理演算法)指定少數幾個「確定是前景」與「確定是背景」的標記，只從這些標記處注水。這徹底馴服了過度分割，是實務上真正可用的版本。它的標準流程會用到第 10 章的型態學運算(侵蝕求前景標記、膨脹求背景標記、距離轉換分離相黏物件)，因此本節也是通往下一章的橋梁。

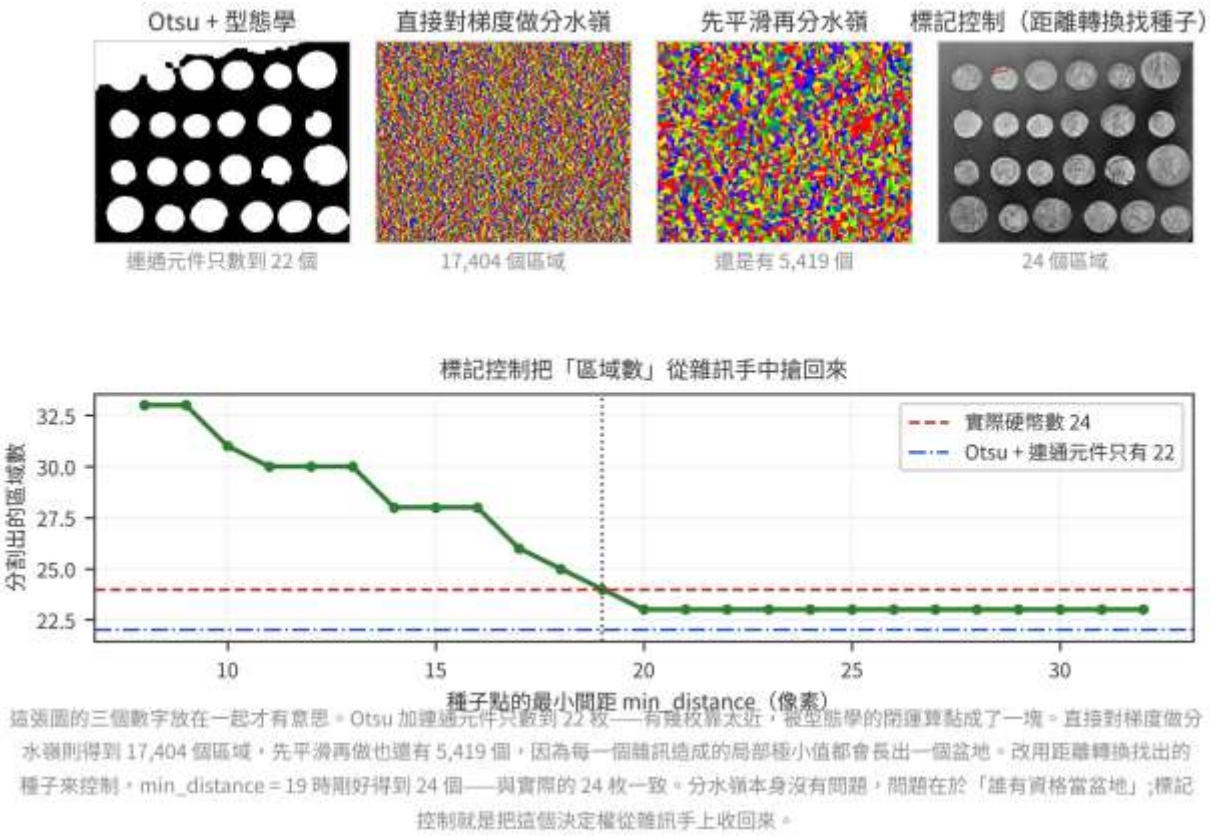


圖 9-19　一萬七千個區域與二十四枚硬幣：標記控制的威力

程式 9-4 標記控制的分水嶺分割

```
import cv2, numpy as np

img = cv2.imread("touching_coins.jpg")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# 1. Otsu 二值化(第 9.2 節)
_, thresh = cv2.threshold(gray, 0, 255,
            cv2.THRESH_BINARY_INV + cv2.THRESH_OTSU)

# 2. 型態學去雜點 + 確定背景(第 10 章)
kernel = np.ones((3, 3), np.uint8)
opening = cv2.morphologyEx(thresh, cv2.MORPH_OPEN, kernel, iterations=2)
sure_bg = cv2.dilate(opening, kernel, iterations=3)

# 3. 距離轉換 → 確定前景(分開相黏物件的關鍵)
dist = cv2.distanceTransform(opening, cv2.DIST_L2, 5)
_, sure_fg = cv2.threshold(dist, 0.7 * dist.max(), 255, 0)
sure_fg = np.uint8(sure_fg)

# 4. 未知區域 = 背景 - 前景
unknown = cv2.subtract(sure_bg, sure_fg)

# 5. 標記並執行分水嶺
n, markers = cv2.connectedComponents(sure_fg)
markers = markers + 1
```

```
markers[unknown == 255] = 0
markers = cv2.watershed(img, markers)
img[markers == -1] = [0, 0, 255]           # 分界線標紅

print("分離出的物件數:", n - 1)
cv2.imwrite("watershed.png", img)
```

9.6 從傳統分割到深度學習語意分割

本章至此介紹的都是傳統方法：它們基於人工設計的規則(門檻、梯度、投票、淹水)，不需要訓練資料，運算量小，可解釋性強。但它們有一個共同的天花板：它們分割的是「低階特徵的一致性」，而不是「語意」。門檻法能把亮的和暗的分開，卻不知道分出來的是貓還是狗;霍夫轉換能找出直線，卻不理解那是車道線還是欄杆。

深度學習帶來了語意分割(Semantic Segmentation)的躍進：為影像中的每一個像素賦予一個類別標籤(這是路面、這是行人、這是天空)。2015 年的 U-Net(原為醫學影像設計)以其對稱的編碼器-解碼器結構與跳接(skip connection)，成為語意分割的里程碑;此後的發展延伸出實例分割(Instance Segmentation，不僅分類每個像素，還區分同類的不同個體，例如畫面中的三個人各自獨立)，代表如 Mask R-CNN;近年更有 SAM(Segment Anything Model)這類基礎模型，能對幾乎任何物件進行提示式分割。第 16 章會實作 YOLO 的分割功能。

那麼傳統方法過時了嗎?並沒有。兩者是互補而非取代的關係：



圖 9-20　傳統方法與深度學習的分工象限

比較面向	傳統分割(本章)	深度學習語意分割
依據	人工規則(灰階、梯度、形狀)	從標註資料學到的語意特徵
訓練資料	不需要	需要大量像素級標註(成本高)
運算量	小，CPU 即可即時	大，通常需要 GPU
可解釋性	高，每步驟皆可理解	低，黑箱
語意理解	無，只分低階特徵	有，能辨識類別
適用場合	受控環境、規律背景、樣本稀少	複雜場景、需要語意、資料充足

表 9-1 傳統分割與深度學習語意分割的比較

工程判斷原則：在受控的工業環境(打光穩定、背景單純)，傳統方法往往又快又準又省——一個 AOI 檢測站用 Otsu 加連通元件就能數清零件，何必動用需要標註數千張影像、還得配 GPU 的深度模型?反之，在開放世界的複雜場景(自駕車看到的街景)，語意的理解無可取代，深度學習才是正解。成熟的工程師懂得在兩者間依場景選擇，甚至混用——例如用深度模型做粗略的語意定位，再用傳統方法做精確的邊界細化與量測。

9.7 應用案例

9.7.1 案例一：零件計數與尺寸量測

這是機器視覺最經典的入門任務，也是本章前五節的綜合演練。情境：一盤散落的螺絲或墊圈，要自動數出數量並量測每一個的尺寸。標準流程串起了本書多章：先用第 5 章的前處理(灰階、CLAHE)改善對比 → 用 9.2 節的 Otsu 門檻二值化 → 用第 10 章的型態學去除雜點、填補孔洞 → 用連通元件標記或輪廓偵測數出物件並計算每個的面積、周長、最小外接矩形。若零件相互接觸，則改用 9.5 節的分水嶺分開它們。這個管線是無數工廠檢測線的骨幹。

9.7.2 案例二：車道線偵測

呼應叫獸開講，這是霍夫轉換最著名的應用。情境：從車前攝影機的影像中即時偵測車道線。完整流程：轉灰階並用第 8 章的顏色資訊強化黃白線 → 用 9.3 節的 Canny 找出邊緣 → 設定「感興趣區域」(ROI，只保留路面梯形區域，排除天空與周邊干擾，這用到第 2 章的 ROI 概念)→ 用 9.4 節的機率霍夫轉換 HoughLinesP 找出線段 → 依斜率把線段分成左右車道並各自擬合成一條線 → 判斷車輛是否偏離。

這個案例也適合討論傳統方法的極限：在標線清晰、光線良好的高速公路上，霍夫轉換又快又可靠;但遇到標線磨損、雨天反光、強烈陰影、彎道(直線模型失效)時就力有未逮——這正是為什麼當代自駕系統改用深度學習來做車道線與可行駛區域的語意分割。從霍夫轉換到深度學習，一部車道偵測的演進史，恰好就是本章 9.1 到 9.6 節的縮影。

9.7.3 案例三：醫學影像的病灶分割

分割在醫療領域至關重要：從 X 光、CT、MRI 中把腫瘤、器官、血管的範圍精確圈出，是後續量測體積、規劃手術、追蹤療效的基礎。傳統上，醫師或工程師會用區域成長(從醫師點選的種子開始擴張)、門檻法(對造影劑增強的區域)、或分水嶺法來輔

助分割。

但醫學影像分割也最能凸顯分割品質的重量。一個腫瘤邊界若分割得太大，可能導致切除過多健康組織;太小則可能殘留病灶。這裡有兩個貫穿本書的呼應：其一，第 7 章談過的還原品質直接影響此處——雜訊與模糊會讓邊界偵測失準;其二，第 7 章「還原 vs. 腦補」的倫理警覺在此同樣適用——若用深度學習模型自動分割病灶，模型「腦補」出的邊界是否可信?正因如此，醫學影像的自動分割結果通常僅作為輔助，最終仍須醫師確認。U-Net 當年正是為醫學影像分割而生，如今深度學習與傳統方法在此領域廣泛並用：深度模型提供快速的初步分割，傳統方法(如主動輪廓、分水嶺)做精細的邊界調整。

9.7.4 案例四：視訊中的移動物件分割

前三個案例都是分割單張影像，第四個案例引入時間維度，銜接第 12 章。情境：固定的監視攝影機，要自動框出畫面中移動的人或車。核心思想是背景相減(Background Subtraction)：先學習一個「沒有移動物件時」的背景模型，再把每一影格與背景相減，差異顯著之處就是前景(移動物件)。

這其實是一種特殊的分割：它不靠灰階、顏色或邊緣，而是靠「時間上的變化」來區分前景與背景。得到前景遮罩後，同樣要用第 10 章的型態學清理雜點、用連通元件標記框出各個物件。OpenCV 提供 MOG2、KNN 等背景相減器，能自適應地更新背景模型(應付光線漸變、樹葉搖動等干擾)。這個案例是智慧監控、人流計數、交通流量分析的基礎，第 12 章會深入。

9.8 本章與全書的觀念連結總表

相關章節	共用的觀念	分割中的角色
第 5 章 空間域強化	直方圖、CLAHE、高斯平滑	分割前處理;Canny 首步的平滑;門檻依直方圖選

第 6 章 頻率域	低通、高通、邊緣即高頻	Canny 內含低通平滑加高通求梯度的雙重濾波
第 7 章 影像還原	雜訊與模糊的影響	雜訊生假邊、模糊使邊緣定位偏移
第 8 章 色彩處理	HSV 顏色分割、Lab 色差	彩色影像常以顏色為分割依據
第 10 章 二值影像	型態學、連通元件、距離轉換	門檻輸出的直接下游;分水嶺標記的前處理
第 12 章 視訊處理	背景相減、移動偵測	以時間變化做前景分割
第 13 章 特徵擷取	輪廓、形狀特徵、SIFT	分割出的輪廓供形狀分析;LoG 為 SIFT 前身
第 16 章 物件偵測	語意分割、實例分割	U-Net、Mask R-CNN 為傳統分割的深度學習後繼

表 9-2 第 9 章與全書其他章節的觀念連結

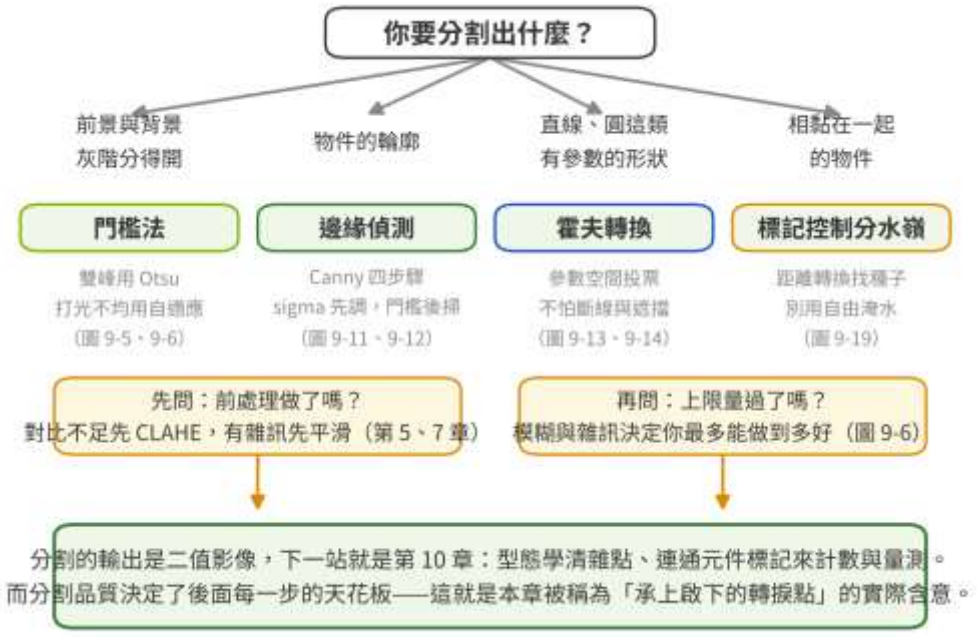


圖 9-21　第 9 章的決策地圖

程式實作 Lab 9

Lab 9-1 三種門檻法的擂台

目標：親手體會各門檻法的適用時機。(a)準備三張影像：對比良好且打光均勻的、打光不均(一側有陰影)的、以及含雜訊的;(b)對每張分別套用全域固定門檻、Otsu、自適應門檻，製成 3×3 的結果對照表;(c)畫出每張影像的直方圖，說明 Otsu 選出的門檻落在哪裡、為什麼在打光不均的影像上失準;(d)對含雜訊影像，比較「直接 Otsu」與「先高斯模糊再 Otsu」的差異;(e)結論：寫出一張決策流程圖——拿到影像後，如何依直方圖形狀與打光狀況選擇門檻法?

Lab 9-2 邊緣偵測參數探索器

目標：掌握 Canny 的參數行為。(a)以 cv2.createTrackbar 建立可即時調整 Canny 高低門檻與高斯 sigma 的介面;(b)觀察並記錄：sigma 太小、太大分別發生什麼事(對照第 6 章「sigma 決定訊號與雜訊的分界頻率」);高低門檻的比例(常建議 1:2 或 1:3)如何影響邊緣的完整與雜訊;(c)實作程式 9-2 的「中位數自動門檻」，與手動最佳值比較;(d)把 Sobel、LoG、Canny 三者的結果並排，說明為何 Canny 的邊緣最俐落(提示：非極大值抑制與遲滯連接)。

Lab 9-3 車道線偵測器

目標：完成叫獸開講的實戰。(a)取一段行車影像(或單張路面照片)，依 9.7.2 節流程：Canny → ROI 遮罩 → HoughLinesP;(b)依斜率把線段分為左右車道，各自用最小平方法擬合成一條直線並延伸繪製;(c)在畫面標出車道中心與車輛中心的偏差，模擬車道偏離警示;(d)測試極限：找出標線磨損、陰影、彎道的影像，記錄霍夫轉換在什麼情況下失效;(e)討論題：承(d)，為什麼當代自駕系統要改用深度學習語意分割?請對照表 9-1 說明。

Lab 9-4 相黏零件的分水嶺分割

目標：解決門檻法的痛點。(a)拍攝或取得一張多個相互接觸的零件(或硬幣、豆子)影像;(b)先用 Otsu 二值化，用連通元件標記計數，觀察相黏的物件如何被錯數成一個;(c)實作程式 9-4 的標記控制分水嶺，比較計數的正確性;(d)調整距離轉換的門檻(0.5、0.7、0.9 倍最大值)，觀察對分離效果的影響;(e)挑戰題：若不用標記控制、直接對梯度影像做分水嶺，會發生什麼?請截圖說明「過度分割」現象。

Lab 9-5 分割方法的擂台與上限量測

目標：建立「先量上限、再比方法」的工程習慣。(a)自製一個合成測試場景：取一張二值遮罩（可用 skimage.data.horse 或自己畫幾何圖形）當作正確答案，渲染成灰階後加上可控的高斯模糊與高斯雜訊，再乘上一個平滑的打光不均場;(b)寫一個 iou(pred, gt) 函式，並對固定門檻、Otsu、自適應門檻三種方法評分;(c)量出上限——把打光場移除、用事後搜尋的最佳固定門檻，記下這個分數，它代表模糊與雜訊造成的不可回復損失;(d)掃描自適應門檻的 blockSize 與 C，畫出 IoU 熱圖，標出落在上限 95% 以內的可用區;(e)把模糊的 sigma 由 0.5 調到 3.0 重做一次，畫出「上限 vs 模糊程度」的曲線，並寫出 300 字的結論：在你的場景裡，改善鏡頭與改善演算法各能帶來多少回報。

Lab 9-6 相黏物件的計數：三種方法的誠實比較

目標：完整走過本章後半的工具鏈，並學會不用標註也能驗證的技巧。(a)拍攝或取得一張含多個相互接觸物件的影像（硬幣、豆子、螺絲、細胞皆可），並以肉眼清點出正確的數量;(b)方法一：Otsu + 型態學 + 連通元件標記，記下數到幾個，並把相黏而被數成一個的位置圈出來;(c)方法二：直接對梯度影像做分水嶺（不給標記），記下區域數——你應該會得到一個大得荒謬的數字，想想為什麼;(d)方法三：距離轉換找局部極大值當種子、再做標記控制分水嶺，並把 min_distance 由 5 掃到 40，畫出「區域數 vs min_distance」曲線，在圖上標出正確答案那條水平線;(e)進階：若物件大小不一，固定的 min_distance 會失效——請改用 h-maxima（先把距離圖減去 h

再取區域最大值）或依物件尺寸自適應的種子策略，並比較兩者的穩健度;(f)寫出結論：在你的影像上，哪一種方法最穩定?若要部署到產線，你會怎麼設計一個「數錯了會自己發現」的檢查機制?

本章重點整理

- 影像分割把影像劃分為有意義的區域，是從「像素」到「區域/物件」的轉捩點;兩大策略為基於不連續性(邊緣、霍夫)與基於相似性(門檻、區域成長、分水嶺)，兩者是一體兩面。
- 門檻法：全域固定門檻適合對比良好且打光均勻者;Otsu 以最大化類間變異數自動選門檻，前提是直方圖雙峰;自適應門檻依局部統計量逐像素定門檻，專治打光不均。
- 邊緣偵測基於微分：Sobel 為帶平滑的一階梯度(方向性高通);Laplacian 為二階微分找零交越但對雜訊敏感，LoG 先平滑再微分改善之;Canny 以高斯平滑、梯度、非極大值抑制、雙門檻遲滯四步驟成為黃金標準。
- 霍夫轉換將「找形狀」翻轉為「參數空間投票」：直線用極座標 $\rho = x\cdot\cos\theta + y\cdot\sin\theta$ 避免斜率無限大，共線點的正弦曲線交會於累加器同一格;霍夫梯度法利用梯度指向圓心的性質加速圓偵測。
- 區域成長從種子出發併入相似鄰居;分水嶺法以「地形淹水」分開相黏物件，但易過度分割，須用標記控制(結合第 10 章的型態學與距離轉換)馴服。
- 深度學習語意分割(U-Net)、實例分割(Mask R-CNN)、基礎模型(SAM)能理解語意，但需大量標註與算力;傳統方法在受控環境、樣本稀少時仍是首選，兩者互補而非取代。

習題

1. 影像分割為什麼被稱為從低階視覺到中階視覺的轉捩點?請以「像素」與「區域」的概念說明。

> *提示與參考解答：分割之前(第 1 到 8 章)處理的是「像素進、像素出」的低階運算(強化、還原、色彩);分割把「屬於同一物件的像素」界定出來，關注點從像素升級到區域與物件，進入中階視覺，並為高階的物件辨識(這是什麼)提供基礎。沒有分割，電腦看到的只是一片數字。*

2. 請比較「基於不連續性」與「基於相似性」兩大分割策略，各舉兩種方法，並說明為何說兩者是一體兩面。

> *提示與參考解答：不連續性著眼區域間的界線(灰階劇變處)，方法有邊緣偵測、霍夫轉換。相似性著眼區域內的一致性(灰階/顏色/紋理相似)，方法有門檻法、區域成長、分水嶺。一體兩面：區域的邊界正是相似性中斷之處，相似的內部正是被邊界包圍的範圍;強健系統常兩者並用。*

3. Otsu 法的目標函數是什麼?它有什麼前提假設?在什麼情況下會失準，該如何補救?

> *提示與參考解答：目標是找出使「類間變異數」最大(等價於類內變異數最小)的門檻，讓前景背景兩群各自集中、彼此遠離。前提是直方圖雙峰。失準情況：前景面積極小(直方圖被背景單峰主宰)、前景背景灰階重疊嚴重。補救：先做前處理拉開對比，或改用自適應門檻。*

4. 為什麼打光不均的影像不能用單一全域門檻?自適應門檻如何解決?這與第 8 章的哪個現象相呼應?

> *提示與參考解答：打光不均使同一物體在亮區與暗區的灰階不同，任何單一門檻都會顧此失彼(亮區背景被誤判為前景，或暗區前景被誤判為背景)。自適應門檻為每個像素依其鄰域局部統計量各自計算門檻，亮區用高門檻、暗區用低門檻。這是第 8 章「洋裝效應/色彩恆常性」在灰階世界的翻版——局部脈絡決定判讀。*

5. 說明梯度大小與方向的幾何意義。Sobel 運算子除了求梯度，還兼具什麼功能?從第 6 章的觀點，Sobel 是什麼濾波器?

> *提示與參考解答：梯度大小代表灰階變化的劇烈程度，方向指向變化最快的方向(垂直於邊緣走向)。Sobel 在求某方向差分時，對垂直方向做加權平均，兼具抗雜訊(平滑)功能。從頻率域看，Sobel 是帶有平滑效果的方向性高通濾波器。*

6. 一階微分與二階微分在邊緣處各有什麼表現?為什麼 LoG 要先做高斯平滑?

提示與參考解答：一階微分在邊緣處產生極值(峰);二階微分在邊緣處產生零交越(由正轉負處即邊緣)。二階微分對雜訊比一階更敏感，故 LoG 先用高斯平滑(低通)壓制雜訊，再用拉普拉斯(高通)找零交越，是「低通加高通」的組合。

7. 請依序說明 Canny 邊緣偵測的四個步驟，並指出每一步對應本書前面的哪個觀念。

提示與參考解答：一、高斯平滑(第 5 章低通，壓制雜訊避免微分放大);二、Sobel 計算梯度(9.3.1 節);三、非極大值抑制(沿梯度方向只留局部最大，細化成單像素邊);四、雙門檻與遲滯連接(強邊必留、弱邊僅在連接強邊時保留，兼顧去假邊與防斷裂)。sigma 是最關鍵參數。

8. 霍夫轉換的核心思想是什麼?為什麼直線偵測要用極座標式 ρ = x·cosθ + y·sinθ 而非斜截式 y = mx + b?

提示與參考解答：核心是觀點翻轉：影像空間中每個邊緣點，對應參數空間中一條曲線(所有可能通過它的直線)，共線的點其曲線交會於一點，故「找直線」變成「參數空間找得票最高的格子」(投票/累加器)。用極座標是因為斜截式無法表示垂直線(斜率無限大)，極座標式對任何方向的直線都有界。

9. 霍夫梯度法如何加速圓偵測?它利用了什麼幾何性質?

提示與參考解答：圓需三參數(圓心 a、b 與半徑 r)，直接投票是三維、成本高。霍夫梯度法利用「圓上每點的梯度方向指向圓心」的性質，讓每個邊緣點只沿自己的梯度方向投票，票數集中到真正圓心;先在二維定出圓心，再回頭決定半徑，把三維投票拆成兩個低維步驟。

10. 分水嶺法的「淹水」比喻是什麼?它擅長解決什麼門檻法的痛點?又有什麼副作用，如何解決?

提示與參考解答：把灰階(或梯度)影像視為地形，灰階為海拔，從各盆地最低點注水，兩湖相遇處築壩即為分割邊界。它擅長分開「相互接觸黏在一起」的物件(門檻法會當成一個)。副作用是過度分割(每個雜訊造成的局部極小值都成一個盆地)。解決：標記控制分水嶺，只從指定的前景/背景標記注水，標記由型態學與距離轉換(第 10 章)產生。

11. 傳統分割與深度學習語意分割最根本的差異是什麼?請各舉一個「該用傳統方法」與「該用深度學習」的場景並說明理由。

    提示與參考解答：根本差異：傳統方法分割「低階特徵的一致性」不懂語意、不需訓練、運算小、可解釋;深度學習分割「語意」能辨識類別但需大量標註與算力、可解釋性低。該用傳統：受控工業環境(打光穩定、背景單純)的 AOI 零件計數，又快又省。該用深度學習：自駕車的開放街景，需要理解「這是行人、這是路面」的語意。可對照表 9-1。

12. (綜合題)請設計一條「散落零件自動計數與尺寸量測」的完整處理管線，列出每一步用到的技術與對應章節，並說明若零件相互接觸該如何處理。

    提示與參考解答：管線：一、前處理——灰階化、CLAHE 增強對比(第 5 章);二、二值化——Otsu 門檻(9.2 節)，必要時先高斯模糊去雜訊;三、清理——型態學開運算去雜點、閉運算填孔(第 10 章);四、標記量測——連通元件標記或輪廓偵測，計算面積、周長、最小外接矩形(第 10、13 章)。若零件相黏：改用標記控制分水嶺(9.5 節)，以距離轉換分離，避免被錯數為一個物件。

13. 「Otsu 就是自動找直方圖的谷底」——這句話在什麼條件下成立?請由類間變異數的公式說明，並估計在前景背景面積懸殊時門檻會往哪邊偏。

    提示與參考解答：類間變異數為 $w0 \cdot w1 \cdot (m0 - m1)^2$，除了「兩群離多遠」之外還乘上「兩群各佔多少」的 $w0 \cdot w1$，而 $w0 \cdot w1$ 在兩群面積相等時最大。因此只有在前景與背景面積相當時，Otsu 的解才會落在谷底;面積懸殊時門檻會被推向「讓兩群面積更接近」的方向，也就是往大的那一群的方向移動。圖 9-3 的實測：硬幣影像的谷底在 139，Otsu 卻選 107，差了 32 階——而且從二值化結果看，Otsu 的選擇反而比較好。

14. 一個檢測任務的前景物件只佔畫面的 1%。直接用 Otsu 會發生什麼事?請提出兩種補救方式，並說明為什麼有效。

    提示與參考解答：直方圖幾乎完全被背景的單峰主宰，前景那個峰小到看不見，Otsu 會把門檻切在背景分布的內部，分割結果幾乎全錯。圖 9-5 的實測顯示，前景面積低於約 3% 時 IoU 由 0.99 急遽崩潰到 0.01，而且是突然崩掉、不是慢慢變差。補救一：先縮小 ROI（用粗定位把視野框到物件附近），讓兩個峰的高度回到同一個量級。補救二：改用自適應門檻或局部對比的方法，讓每個像素依鄰域決定門

檻，不受全域分布比例的影響。

15. 自適應門檻的 blockSize 與 C 各該怎麼選?請說明它們的物理意義，並描述設錯時各會出現什麼症狀。

    提示與參考解答：blockSize 是估計「局部背景亮度」所用的鄰域邊長，必須大於你要抓的前景結構寬度（否則前景自己會把局部平均拉高，造成筆畫內部被判成背景、只剩輪廓），又必須小於光照變化的空間尺度（否則等於退化成全域門檻）。C 是從局部平均扣掉的常數，相當於要求前景比鄰域平均再暗（或亮）多少，用來排除平坦區的雜訊;C 太小會讓平坦區的雜訊整片翻成前景。圖 9-7 在合成文件上掃過兩個參數：最佳為 blockSize 71、C 26，而落在最佳值 95% 以內的組合只佔全部 357 種的 14%。

16. 什麼是「效能上限」?為什麼在評估一個分割方法之前應該先把它量出來?請以圖 9-6 的數據說明。

    提示與參考解答：上限是指在同樣的成像條件（模糊、雜訊、量化）下，任何方法都不可能超過的成績。量法：把干擾因素之一（此例為打光不均）移除，再用事後挑的最佳參數，得到的分數就是上限。圖 9-6 的實測：把打光弄成完全均勻、再用最佳固定門檻，IoU 也只有 0.81，剩下的損失來自鏡頭模糊與雜訊。調過參數的自適應門檻拿到 0.78，已經是上限的 95%。若不先量上限，看到 0.78 很容易以為還有很大的改善空間，於是把時間花在演算法上;事實上真正該改的是鏡頭與曝光。

17. 一階微分與二階微分在「階躍邊緣」與「細線」上的表現各是什麼?為什麼說拉普拉斯會把一條線報成兩條邊?

    提示與參考解答：階躍邊緣的一階微分出現單一極值，二階微分出現一個零交越，兩者都正確指出邊緣位置。但一條細線（屋脊）的剖面是「升上去再降下來」，它的二階微分在線的兩側各有一個零交越（圖 9-8 第三欄實際畫出 @95 與 @97），所以拉普拉斯會輸出兩條邊而不是一條線。這不是雜訊造成的，而是二階方法在細結構上的固有行為;要找線狀結構應改用專門的線偵測器（如 Frangi 濾波）或先做骨架化。

18. 從頻率域的觀點，Sobel 是什麼樣的濾波器?請說明它為什麼比單純的中央差分 [−1, 0, 1] 更能抗雜訊。

    提示與參考解答：Sobel-x 可分解為水平的 [−1, 0, 1] 與垂直的 [1, 2, 1]，因此它在

水平方向是高通（直流分量的響應為 0，峰值出現在 Nyquist 的 0.5 倍處），在垂直方向卻是低通。圖 9-9 實測：固定在峰值水平頻率上，垂直頻率走到 Nyquist 的一半時，響應由 8.00 掉到 4.05（剩 51%），走到 Nyquist 時恰好歸零——因為 [1, 2, 1] 的頻率響應 2 + 2cos(ω) 在 ω = π 處為零。這個垂直方向的平均正是抗雜訊的來源：單純的中央差分沒有這一項，雜訊會原封不動地被放大。

19. 請由「核係數平方和」推導各邊緣運算子的雜訊放大倍率，並說明為什麼「放大得少」不等於「比較好」。

    提示與參考解答：對獨立雜訊做卷積，輸出標準差等於輸入乘上核係數平方和的平方根（第 7.7.4 節）。Sobel 的係數平方和為 12，倍率 √12 ≈ 3.46;五點拉普拉斯為 20，倍率 √20 ≈ 4.47。圖 9-10 的實測與理論最大只差 0.5%。但 LoG（sigma=2）的倍率只有 0.047，卻不能就此說它比較好——它的係數本來就小，把訊號也一起縮小了。該比的是輸出的訊噪比：以乾淨階躍邊緣的峰值響應除以輸出雜訊，拉普拉斯只有 2.9，LoG（sigma=2）是 18.2，提升 6.3 倍。先平滑再微分之所以划算，是因為它壓掉的雜訊多於訊號。

20. Canny 的非極大值抑制為什麼必須沿著梯度方向進行?它實際上丟掉了多少像素?為什麼這一步是後面遲滯連接能夠運作的前提?

    提示與參考解答：梯度向量永遠垂直於邊緣走向（沿邊緣走灰階不變，方向導數為零），所以「跨過邊緣」的方向就是梯度方向，梯度隆起的剖面正好落在這個方向上。沿此方向與前後兩格比較，只留最大者，就把隆起壓成脊線。圖 9-11 實測：梯度圖中超過低門檻的 31,522 個像素，抑制後只剩 8,127 個，少掉 74%，而邊緣位置不變。這一步是遲滯連接的前提，因為遲滯是用「弱邊有沒有連到強邊」來判斷的;若邊緣是好幾像素寬的粗帶子，連通性就失去鑑別力了。

21. 調 Canny 的參數時，sigma、高門檻、高低門檻比三者的重要性順序是什麼?請以可量化的方式說明，並給出一條實務上的調參流程。

    提示與參考解答：圖 9-12 在一個含高對比物件、低對比物件與背景紋理的合成場景上量測（每個 sigma 都用它自己最好的門檻，才算公平）。結果：四種高低比（1.5:1 到 3:1）各自的最佳 F1 只差 0.028，所以比值幾乎不重要;但高門檻很挑，F1 落在最佳值 95% 以內的只有 72 到 84 這一小段。sigma 則決定了上限本身：sigma 0.6 時召回率只有 0.70（為擋雜訊必須把門檻設高，低對比物件跟著不見），sigma 4.2 時 F1 掉到 0.70（邊緣被平滑掉）。流程：一、先依雜訊程度與最小結構尺寸決

定 sigma;二、固定比值 2:1，掃高門檻;三、必要時再微調比值。

22. 霍夫轉換的累加器需要多大?請分別估計一張 1080p 影像上的直線累加器，以及一張 303×384 影像上半徑不限的圓累加器，並說明這對實務的啟示。

提示與參考解答：直線累加器為 2D × 180 格（D 為對角線長）。1080p 的 D = 2203，累加器 4406 × 180 = 793,080 格，int32 約 3 MB，完全可行。圓累加器為 H × W × R:303 × 384 若半徑不限（1 到 192）就是 22,339,584 格、約 89 MB，而且每個邊緣點要投的票從 180 暴增到數萬。啟示：cv2.HoughCircles 的 minRadius 與 maxRadius 實質上是必填的——把半徑限在 12 到 36，累加器降到 2,908,800 格才跑得動。降維的辦法就是霍夫梯度法：利用「圓上每點的梯度方向通過圓心」，讓每個邊緣點只沿自己的梯度方向投票，先定圓心再定半徑;代價是對梯度方向極度敏感，所以程式 9-3 一定要先做 medianBlur。更一般的原則是成本隨參數個數指數成長：兩個參數很好用，三個要想辦法降維，超過三個就該改用輪廓擬合。

23. 什麼是區域成長的「洩漏」?請說明它的成因，並以可量化的方式描述它發生時有多突然。

提示與參考解答：區域成長從種子出發，把與區域夠相似的鄰居併進來。只要物件邊界上有任何一個像素的灰階落差小於容許值，就形成一個破口，區域會從那裡流進背景，然後整張影像都被併入。圖 9-17 的實測：最佳容許值 42（IoU 0.967），但容許值升到 56 時面積由 11,395 暴增到 117,863（10 倍），IoU 掉到 0.077。它沒有中間地帶，這也是區域成長很少單獨使用的原因;實務上會先用邊緣或型態學把邊界封好（第 10 章），再放它進去長。

24. 分水嶺法的「過度分割」有多嚴重?請說明成因，並說明標記控制為什麼能解決它。

提示與參考解答：分水嶺從每一個局部極小值開始注水，所以分割出的區域數完全等於局部極小值的個數;而真實影像的梯度圖上，雜訊會製造出成千上萬個局部極小值。圖 9-19 在 24 枚硬幣的影像上實測：直接對梯度做分水嶺得到 17,404 個區域，先做 sigma=2 的高斯平滑再做仍有 5,419 個。標記控制的作法是不讓水從所有局部極小值自由湧出，而是用距離轉換找出「每個物件的中心」當作指定的種子;本例在 min_distance = 19 時得到剛好 24 個區域，與實際硬幣數一致。重點是分水嶺本身沒有問題，問題在於「誰有資格當盆地」——標記控制就是把這個決定權從雜訊手上收回來。

25. (綜合設計題)請為一條「散落零件自動計數與尺寸量測」的產線設計完整管線，列出每一步的技術、對應章節與參數決定方式，並說明如何處理零件相互接觸、以及如何驗收這套系統。

    提示與參考解答：一、光源與成像：封閉光箱、背光（讓零件成為高對比剪影），關閉自動曝光與自動白平衡（第 8 章）——這一步決定了後面所有步驟的上限。二、前處理：灰階化，必要時 CLAHE 拉對比（第 5 章）;若有脈衝雜訊先用中值濾波（第 7 章）。三、二值化：背光下直方圖是乾淨雙峰，用 Otsu;若打光仍有梯度則改自適應門檻（圖 9-6、9-7）。四、清理：型態學開運算去雜點、閉運算補洞（第 10 章）。五、分離相黏零件：距離轉換 + 標記控制分水嶺（圖 9-19）;關鍵參數 min_distance 應依零件的最小半徑決定，不要盲試。六、計數與量測：連通元件標記（第 10 章）取面積、外接矩形、最小外接圓;以實際尺寸標定像素尺寸（第 4 章）。七、驗收：先量上限——取一批已知答案的樣本，用人工標註算出 IoU 與計數誤差的上限（圖 9-6 的作法），再要求系統達到上限的某個百分比;同時記錄跨班別、跨批次的重複性，一旦漂移就回頭檢查光源而不是調參數。

參考資料與延伸閱讀

- Gonzalez, R. C., & Woods, R. E., Digital Image Processing, 4th ed., Pearson, 2018, Ch. 10(影像分割之標準參考)。
- Hough, P. V. C., Method and Means for Recognizing Complex Patterns, US Patent 3,069,654, 1962.(霍夫轉換原始專利，本章叫獸開講之史料來源)
- Duda, R. O., & Hart, P. E., Use of the Hough Transformation to Detect Lines and Curves in Pictures, Comm. ACM, 1972.(極座標表示法)
- Canny, J., A Computational Approach to Edge Detection, IEEE TPAMI, 1986.(Canny 邊緣偵測原始論文)
- Otsu, N., A Threshold Selection Method from Gray-Level Histograms, IEEE Trans. SMC, 1979.
- Ronneberger, O., Fischer, P., & Brox, T., U-Net: Convolutional Networks for Biomedical Image Segmentation, MICCAI, 2015.

- 補充教材與課程資訊：@prof(https://page.line.me/prof)